

ENHANCING STOCK MARKET INSIGHTS THROUGH HYBRID AI MODEL DRIVEN TECHNICAL ANALYSIS

*Project report submitted to
Visvesvaraya National Institute of Technology, Nagpur
in fulfillment of requirement for the award of
degree of*

Master of Technology
in
Applied Artificial Intelligence

by
Adwait Shirish Sonawane (MT23AAI031)

under the guidance of
Dr. Ankit Bhurane



Department of Applied Artificial Intelligence
Visvesvaraya National Institute of Technology, Nagpur
Nagpur 440010 (India)

December, 2025

ENHANCING STOCK MARKET INSIGHTS THROUGH HYBRID AI MODEL DRIVEN TECHNICAL ANALYSIS

*Project report submitted to
Visvesvaraya National Institute of Technology, Nagpur
in fulfillment of requirement for the award of
degree of*

Master of Technology
in
Applied Artificial Intelligence

by
Adwait Shirish Sonawane (MT23AAI031)

under the guidance of
Dr. Ankit Bhurane



Department of Applied Artificial Intelligence
Visvesvaraya National Institute of Technology, Nagpur
Nagpur 440010 (India)

December, 2025

Copyright © Visvesvaraya National Institute of Technology, Nagpur(VNIT)
December, 2025



DECLARATION

I, "Adwait Shirish Sonawane" hereby declare that the project work titled "**Enhancing Stock Market Insights Through Hybrid AI Model Driven Technical Analysis**" is carried out by me in the Applied Artificial Intelligence of VNIT, Nagpur, India. The work is original and has not been submitted earlier as a whole or in part for the award of any degree/diploma at this or any other Institution/University.

Adwait Shirish Sonawane
MT23AAI031

Date: December 01, 2025

CERTIFICATE

This is to certify that the thesis titled "**Enhancing Stock Market Insights Through Hybrid AI Model Driven Technical Analysis**" submitted by **Adwait Shirish Sonawane** in partial fulfilment of requirements for the award of Master of Technology in the Applied Artificial Intelligence of VNIT, Nagpur. The work is comprehensive, complete and fit for evaluation.

Dr. Ankit Bhurane
Assistant Professor,
Applied Artificial Intelligence, VNIT, Nagpur

Dr. Ashwin Kothari
Professor and Head,
Applied Artificial Intelligence, VNIT, Nagpur
Date: December 01, 2025

NON PLAGIARISM REPORT



Page 2 of 86 - Integrity Overview

Submission ID trn:oid::1:3435435022

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **107 Not Cited or Quoted 9%**
Matches with neither in-text citation nor quotation marks
-  **1 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 7%  Internet sources
- 6%  Publications
- 4%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Figure 1: Plagiarism Report

ACKNOWLEDGEMENTS

My sincere gratitude to my guide, for his constant support and encouragement, without which the project would not have been possible. My heartfelt gratitude to him for offering all of the facilities and assisting me in overcoming obstacles.

I would like to thank all the professors of ECE department department for helping me by some way or the other. I sincerely thank our project evaluators for their valuable comments and suggestions. I thank Head of ECE department for giving an opportunity for the project.

Date: December 01, 2025

Adwait Shirish Sonawane(MT23AAI031)

Abstract

Stock price forecasting is a challenging but rewarding problem due to the inherent challenges involved like the volatility, nonlinearity and dynamic nature of financial markets. Stock forecasting might involve trend forecasting or as I explore in this project forecasting based on short term price movement to predict the next day's closing. Traditional Technical analysis and their use in creating forecasting statistical models provide a solid base and robust techniques which could be included in the modern technical analysis using machine learning. Even so the traditional forecasting models lack adaptability to ever changing market behaviour and also are limited by manual capacity to interpret complex temporal patterns. This limitation can be eliminated by using the Machine Learning and deep learning approach for creating sophisticated models that can better understand today's complex market environment. This this project I have applied machine learning and deep learning techniques to predict next day's closing price based on the short-term data.

The last two years of stock data was collected through Kite Connect API for companies listed on NSE. This data is divided into NIFTY50 companies, NIFTY200 companies and the broader NSE universe. Dataset collected from Kite connect API contains Open High Low Close and Volume data. This data then can be pre-processed to calculate additional Technical indicators that indicate market volatility, and trends. Technical Indicators like Relative Strength Index(RSI), On Balance Volume(OBV), Exponential Moving Average(EMA) and Average True Range (ATR) are calculated and used as additional features that enhance the market movement and help models better learn the patterns.

In this project I have implemented simple models like GRU and LSTM, but also trained larger models with transformers and a hybrid model with CNN-LSTM method. The models are trained with the time-series split to preserve temporal data integrity. The model's performance are assessed with Mean Absolute Error and Root Mean Square Error(RMSE). Implemented code to retrieve data from kite, preprocess the data for training and then show the results according to the model selected. The major findings from the project are smaller models like GRU performed best with limited data, like with single company's historical data. LSTM performed well overall with limited single company as well as multi company data like NIFTY200. Lastly models trained on CNN-LSTM and Transformers needed larger dataset to be trained.

List of Figures

1	Plagiarism Report	ii
3.1	System Flow	19
3.2	Architecture of Data Flow.	28
5.1	Architecture of the GRU block.	37
5.2	Architecture of the LSTM block.	41
5.3	Architecture of the Transformer block.	47
5.4	Architecture of the CNN + LSTM block.	53
5.5	Conceptual Diagram of a CNN-LSTM Model Architecture	54
6.1	GRU without Indicators	57
6.2	GRU with Indicators Validation Behaviour	57
6.3	LSTM without Indicators Validation Behaviour	58
6.4	LSTM with Indicators Validation Behaviour	58
6.5	Transformers without Indicators Validation Behaviour	59
6.6	Transformers with Indicators Validation Behaviour	59
6.7	Regression Models on Single Company Data	60
6.8	Regression Models on Single Company Data With Indicators	61
6.9	Regression Models Compared Without Indicators Included	61
6.10	Regression Models Compared with Indicators Included	62
6.11	Prediction Errors for Regression Models on Single Company Data	62
6.12	GRU with stock data	64
6.13	GRU with Indicators included (EMA (12, 26), RSI (14), ATR (14), OBV)	64
6.14	LSTM with stock data	65
6.15	LSTM with Indicators (EMA (12, 26), RSI (14), ATR (14), OBV)	65
6.16	Transformers with stock data	66
6.17	Transformers with Indicators (EMA (12, 26), RSI (14), ATR (14), OBV)	66
6.18	CNN-LSTM with stock data	67

6.19	CNN-LSTM with Indicators (EMA (12, 26), RSI (14), ATR (14), OBV)	67
6.20	CNN-LSTM with Indicators (Close, MA5, MA20)	68
6.21	CNN-LSTM with Indicators (Close, EMA10, STD5)	68
6.22	CNN-LSTM with Indicators (Close, Return, STD20)	69
6.23	CNN-LSTM with Indicators Validation Loss Comparisons	69

List of Tables

2.1	Summary of Existing Machine Learning, Deep Learning, and Hybrid Approaches in Stock Market Forecasting	13
2.2	Comparison of machine learning, hybrid deep learning, and emerging AI approaches for stock market prediction.	15
3.1	Dataset Categorization Based on Usage and Scope	19
3.2	Dataset Splitting Strategy	28
3.3	Model Hyperparameters	29
3.4	Model Evaluation Metrics	29
4.1	Recommended Hardware Specifications	30
4.2	Project Technology Stack	31
5.1	Key Notations for Gated Recurrent Unit (GRU) Equations	37
5.2	Assessment of Model Characteristics (likely GRU/Simple RNN variant)	38
5.3	Model Architecture Summary	39
5.4	Summary of Technical Indicators	40
5.5	Stacked GRU Model Architecture Summary	40
5.6	Key notations and equations for the Long Short-Term Memory (LSTM) network	43
5.7	Assessment of LSTM Model Characteristics	44
5.8	Stacked LSTM Model Architecture Summary	45
5.9	Comparison of GRU vs. LSTM Architectures	45
5.10	Stacked LSTM Model Architecture Summary	46
5.11	Key notations and equations for the Transformer encoder layer used in this project	49
5.12	Analysis of a Deep Learning Model (Likely Transformer/Self-Attention-based) for Time Series Prediction	50
5.13	Model Architecture Summary	51

5.14	Model Architecture Summary	51
5.15	Comparison of Recurrent Neural Network (RNN) Architectures . . .	52
5.16	Analysis of a Hybrid Deep Learning Model (Likely CNN-LSTM) . . .	54
5.17	Model Architecture Summary	55
5.18	Model Architecture Summary (Likely CNN-LSTM)	55
6.1	Comparative Model Performance Across Datasets	63
6.2	Feature Set Performance Comparison	68
6.3	Strengths and Measures of Technical Indicator Sets	70
6.4	Summary of Technical Indicator Sets	70

Contents

Abstract	iv
List of Figures	v
List of Tables	vii
1 Introduction	1
1.1 Introduction and Background	1
1.2 Evolution of Technical Analysis	1
1.3 Limitations of Classical Forecasting Models	3
1.4 Machine Learning as a Solution	3
1.5 Importance of Technical Indicators	4
1.6 Need for Hybrid Intelligent Systems	5
1.7 Identified Research Gap	6
1.8 Problem Statement	7
1.9 Research Objectives	7
1.10 Research Questions	8
1.11 Scope, Limitations, and Assumptions	8
1.12 Novelty and Contributions	9
2 Literature Review	10
2.1 Stock Market Forecasting and Technical Analysis	10
2.1.1 Introduction	10
2.1.2 Classical Forecasting Techniques	10
2.1.3 Traditional Trading Strategies	11
2.1.4 Traditional Machine Learning models in Forecasting	11
2.2 AI Driven Stock Forecasting	12
2.3 Comparison Between different Model and research	13
2.4 Key Findings, Strengths, and Limitations	15

2.5	Evaluation Challenges, Research Gaps and Inconsistencies	17
3	Methodology	18
3.1	Overview	18
3.2	System Workflow	18
3.3	Data Acquisition	19
3.4	Data Preprocessing	20
3.4.1	Handling Missing Values	20
3.4.2	Scaling and Normalization	22
3.5	Feature Engineering	22
3.5.1	Raw Feature Selection	22
3.5.2	Windowing Strategy	23
3.5.3	Training Test Validation Structuring	23
3.5.4	Indices Used	24
3.6	Technical Indicators	24
3.6.1	Exponential Moving Average (EMA)	24
3.6.2	Relative Strength Index (RSI)	25
3.6.3	Average True Range (ATR)	26
3.6.4	On-Balance Volume (OBV)	27
3.6.5	Summary of Indicator Contribution	27
3.7	Model Training and Validation Datasets	27
3.8	Prediction and Deployment Pipeline	27
3.9	Architecture diagram of data flow	28
3.10	Hyperparameter Configuration	28
3.11	Evaluation Metrics	28
4	Experimental Setup and Implementation	30
4.1	Overview	30
4.2	Hardware Configuration	30
4.3	Software Stack	31
4.4	Kite Connect API	31
4.4.1	Developer Account	32
4.5	Authentication process	32
4.6	Getting Historical Data	33
4.6.1	Historical Data Endpoint	33
4.6.2	Path Parameters	33
4.6.3	Query Parameters	33

4.6.4	Stock Data Example	34
5	Model Details	36
5.1	Gate Recurrent Unit (GRU) Models	36
5.1.1	Introduction to GRU Architecture	36
5.1.2	Mathematical Model for GRU	37
5.1.3	Advantages and Disadvantages of GRUs Category Details . . .	38
5.1.4	Use of GRU in Project	38
5.1.5	GRU Model training Configuration	39
5.1.6	Summary of GRU model Performance	40
5.2	Long Short-Term Memory (LSTM) Model	41
5.2.1	Overview of LSTM Architecture	41
5.2.2	Why Use LSTM in time series forecasting	42
5.2.3	Mathematical Model for LSTM	42
5.2.4	Advantages and Disadvantages of LSTM	43
5.2.5	Role of LSTM in this Project	43
5.2.6	Trained LSTM Model Configurations	45
5.2.7	Performance Summary	46
5.3	Transformer Model	47
5.3.1	Transformer Architecture	47
5.3.2	Motivation for Using Transformer in this project	47
5.3.3	Mathematical Model for Transformer Encoder	48
5.3.4	Advantages and Disadvantages of Transformers	50
5.3.5	Role of Transformers in This Project	50
5.3.6	Trained Transformer Model Configurations	50
5.3.7	Performance Summary	51
5.4	Conv1D + LSTM Hybrid Model	53
5.4.1	Introduction to the Hybrid Architecture	53
5.4.2	Motivation for Using a Hybrid Model in Time-Series Forecasting	53
5.4.3	Advantages and Disadvantages of the Hybrid Model	54
5.4.4	Trained Configurations of the Hybrid Model	54
6	Results and Analysis	56
6.1	Overview	56
6.2	Training and Validation Behavior	56
6.3	Single Company Data Used	60
6.3.1	Quantitative Evaluation Results	63

6.4	Effect of Feature Engineering vs. Raw OHLCV Inputs	63
6.5	CNN-LSTM Feature Set Performance	68
6.5.1	Indicators tested and their purpose	70
6.6	Generalization vs. Specialization Behavior	71
7	Summary and Conclusions	72
7.1	Summary of Work	72
7.2	Problem Statement and Solution	72
7.3	Findings and thier Significance	73
7.4	Limitations of the Project	74
7.5	Future Work	74
7.6	Conclusion	75
	Annexure	76
A.	Dataset and Preprocessing Summary	76
B.	Model Configurations (Supplementary)	78
C.	Implementation and API Summary	80

Chapter 1

Introduction

1.1 Introduction and Background

The first stock exchange was the Amsterdam stock exchange, that was established in 1602 by the Dutch East India company to trade stocks. Since the foundation, stock exchanges have been consistently proven as easy way for people to participate in the economy, and get involved with businesses. This has also helped business raise capital further bringing economic prosperity and creating jobs as well as wealth for investors. It has also helped with innovation as it has helped companies smaller as well as larger to raise money for their new projects, distributing risks and making capital available for development of new products and research.

India's first stock exchange was the Bombay stock exchange (BSE) started in 1875 and is also Asia's oldest stock exchange. Similar to the other stock exchanges in the world the Indian stock market has brought economic prosperity and opportunity for people to invest in the development of Indian companies. It has also help create opportunity for foreign investors to invest in India that has greatly helped Indian companies raise capital and develop new and innovative products for India. There are a number of Stock exchanges in India, but Bombay stock exchange (BSE) and National Stock Exchange (NSE) are the two main and the most important stock exchanges on which most of the companies whose stocks are tradable in India are listed.

1.2 Evolution of Technical Analysis

The exchanges in the past use to maintain physical ledgers of pricing action in the market, and trades used to be done using verbal communication and hand gestures in

an open outcry system to execute trades. Then Manual record keeping was standard. The development of stock tickers in 1860s followed by electronic system in 1960s were major milestones in how the stock market operated and how the users could participate and analyse stocks for trading. The creation of fully electronic exchange like NASDAQ in 1971 opened doors to collection, storage and access of huge amount of stock data that could be used to analyse price action over time. This also meant that it opened doors to analysis of different events further deepening the understanding and development of techniques for forecasting prices. The internet further helps in automation of data flow and improved accessibility as well as improved data quality as other participants could also collect and store data in even more granular detail.

With the emergence of Internet trading and availability of huge past data, gave rise to increased competition and thus research both in Fundamental and Technical analysis. Technical Analysis overtime developed in to an extremely complete, structured and quantitative practice based on mathematical models. The first formalized technical analysis pattern is considered the candlestick technique developed by the Japanese rice merchants Munehisa Homma in the 18th century. The candle stick pattern help tracks and better visualize the open, low, high, close of the time scale decided and also helps recognize emergent patterns formed as a result of plotting the candle stick overtime. These patterns were used then to understand market movement and predict the price action.

Further competitiveness of markets and rise of internet trading, helped develop technical indicators including Simple Moving Average (SMA), Volume based Trend lines, Chart patterns based on Candle stick data, Moving Average Convergence Divergence (MACD), Relative Strength Index (RSI) and are still some of the most popular indicators used in trading today. These indicators helped traders better visualize and understand the impact of different event on price when working with historical data proving that Technical analysis could give meaningful information and thus aid in forecasting prices in the future.

As Technical Analysis is heavily dependent on data and is basically based on developing mathematical model to understand performance of market and forecasting future to maximize probability of good predictions to make profits, use machine learning models became the obvious way forward. AI models are extremely good at adapting mathematical models and are devoid of human biases. They also have good scalability as compared to the number of humans that will be needed to study and interpret data.

1.3 Limitations of Classical Forecasting Models

Technical analysis before Machine Learning method were introduced heavily relied on mathematical and statistical model of time series data such as Moving Average, Oscillators such as RSI and stochastic oscillators, Moving Average Convergence Divergence, Volume Indicators which are effective at identifying baseline patterns. They assume linearity and stationarity of properties which mostly does not match the real work behaviour of the market.

Market data is inherently noisy, nonlinear, non-stationary and chaotic. The market conditions can shift rapidly due to external triggers may it be with news related to the specific company itself or it could be related to larger factors like political or global factors. Classical technical pattern fails to recognise and adapt to these external factors that could have huge movement not seen in historical data.

The prime example of failure of Technical analysis could be seen during the economic downturn during the Covid Pandemic during the period of 2020-2021. This also extends to the rapid recovery of the market in the later years not following the linear growth that the market followed before.

The main reason for the failure of Technical analysis is it treats time series data in isolation rather than larger context, may it be the Fundamentals or the Sentiment changes. Machine learning could solve the issue by creating a model that combines all the three factors Fundamental, sentimental and Technical indicators. In this project though I focus mainly on use of Machine learning to create Technical analysis models as combining all three would create too big of a scope to focus on and explore in detail the impact of Technical indicators and their effectiveness when used to train a machine learning model.

1.4 Machine Learning as a Solution

Machine learning models are excellent at learning relationships from data, and can better accommodate for complex non linear relationships between features. The classical Technical analysis methods rely heavily on linearity of relationship and human intuition to understand these when there are deviations. Thus, machine learning models trained on the huge amount of data now available for the stock can generalize understanding the underlying patterns that human interpreters could miss.

Human interpretation of stock mathematical models has the limitation that they can consider only a limited amount of feature and indicators in combination while making a forecasting prediction making the prediction heavily biased by the signals provided

by those particular indicators. As all Technical indicators have their limitations and combining them could elevate the biases that these limitations might bring, machine learning models become an obvious solution.

Machine learning models, especially deep learning models can consider a huge number of features while training and can also combine different Technical indicators together, thus reducing the impact of each individual Technical indicators. They also can interpret and learn complex relationships between these indicators and their overall impact on the forecasting. Humans on the other hand could miss, ignore or have biases towards considering these indicators.

Traditional machine learning algorithms saw good success in forecasting as Technical analysis was hugely based on mathematical modelling of stock data. Algorithms such as Support Vector machines, Decision Tree, Random Forests, Logistical Regression and even Linear regression gave impressive result while forecasting. Taking the example of SVM models when combined with Technical indicators like Moving Average, Bollinger Bands, RSI, Stochastic Oscillators etc (Indicators help in understand different features of market like price action and volume of trade) gave impressive result achieving up to 77.8% of accuracy, and also demonstrated robustness especially for volatile stocks.

Introduction of neural networks and time series neural networks such as GRU and LSTM gave even better forecasting results. Deep learning used in Technical analysis are even better at forming complex relationship and combining patterns to give very accurate forecasting. They are also capable of including factors like Fundamentals of the company and events like seasonal ness of products sold or announcement of earnings. Even further inclusion of LLM to has enabled the inclusion of sentimental data, closely tracking news and social media to understand and include that factor in the larger models.

Thus, Machine learning has made it possible to integrate Technical, Fundamental and Sentimental data in an automated system, that can consider a huge number of features to effectively predict and forecast prices.

1.5 Importance of Technical Indicators

The raw data that is collected mostly consist of data that is required to create candlestick charts, as it is the most popular form of charts used in Technical Analysis. This data consists of open, close, low, high and also includes the volume of trade. These data features then can further used to calculate Technical Indicators that combine them is a way that creates distinctive feature.

Indicators such as MACD, RSI, and EMA provide diverse patterns, momentum, surges, trend strength, and volatility that enhance the data that the Machine learning algorithms can interpret and form stronger relationship between features. Studies have shown that combining these indicators that target a particular signal each provide better data for algorithm making the algorithm generalize better.

These indicators also reduce noise and fluctuation that might have been caused due to temporary instability in market. Making the supervised learning models and deep learning neural networks trained on this time series data more stable and generalized.

1.6 Need for Hybrid Intelligent Systems

Model Type	Strengths	Weaknesses
CNN (Convolutional Neural Networks)	Extract local patterns and micro-window features, detect volatility spikes, learn shape-based signals (e.g., candlestick formations, breakout patterns, momentum bursts).	Limited memory and poor capability in modeling long-term dependencies or sequential temporal relationships.
LSTM / GRU	Effectively captures long-term temporal relationships and sequential memory, especially useful for time-dependent and autocorrelated stock behavior.	Slow to learn high-frequency short-term patterns and may be sensitive to noise and rapid volatility shifts.
Transformers	Efficient at modeling global time dependencies and attending to the most relevant historical signals, enabling context-aware forecasting.	Requires large training data and is computationally expensive; weaker at extracting local high-frequency features without convolutional or embedded preprocessing.

A hybrid model mitigates this issue by combining the strengths of multiple neural architectures.

- **CNN → LSTM hybrids** First extract short-term patterns using convolutional filters and then feed the learned representations into an LSTM layer to capture longer temporal dependencies.

- **Parallel CNN + LSTM models** Enable the network to independently learn both local feature patterns and long-term sequential structure before merging them at a later stage.
- **Transformer-LSTM hybrids** Use self-attention mechanisms to selectively focus on the most relevant historical signals while preserving sequential memory through recurrent layers.
- **CNN–Transformer–LSTM stacks** Model multi-scale temporal behavior by combining:
 - noise-filtered short-term structure (via CNNs),
 - long-term dependencies (via LSTM),
 - dynamic context selection (via attention in Transformers).

Hybrid architectures improve prediction robustness as:

- Reduce sensitivity to noise by distributing learning across complementary structures.
- Capture multi-scale temporal dynamics rather than forcing one model to learn all patterns.
- Improve generalization across stocks and varying market regimes, particularly in heterogeneous datasets such as broad index constituents.
- More efficiently model non-linear interactions between price, volatility, volume, and technical indicators compared to standalone architectures.

1.7 Identified Research Gap

While existing research demonstrates significant advancements in stock prediction using ML, deep learning, and hybrid models, several gaps remain. First, a large portion of the work focuses on markets such as the NASDAQ, NYSE, and Shanghai Stock Exchange, with far less emphasis on emerging markets such as India. This creates generalization concerns, as financial behavior differs across regions due to regulation, liquidity conditions, economic stability, and trading culture.

Second, many studies prioritize prediction accuracy as the primary evaluation measure. However, high classification accuracy does not always translate to profitable or low-risk trading outcomes, especially when ignoring transaction fees, slippage, market

volatility, or drawdown effects. Recent papers highlight that success metrics should incorporate profitability measures such as return on investment (ROI), maximum drawdown, and the Sharpe ratio.

Third, deep learning approaches, although powerful, are often criticized for their black-box nature, making them difficult to interpret and validate.

This lack of transparency poses challenges in regulated financial environments where decision justification is required.

Finally, while hybrid models show promise, there is limited research on systematically comparing classical ML, deep learning, and hybrid approaches specifically combined with technical analysis inputs under consistent evaluation frameworks especially using Indian market data.

These gaps collectively indicate a need for structured research that evaluates machine learning-driven technical analysis within the context of Indian market behavior while incorporating explainability and meaningful financial evaluation metrics.

1.8 Problem Statement

Based on the identified research gap, the key challenges addressed in this thesis are, traditional stock forecasting methods and standalone technical analysis approaches are limited in their ability to model nonlinear, volatile, and complex stock price behavior, particularly in emerging markets such as India. While machine learning and deep learning approaches have shown promising results, there is insufficient structured evaluation of their effectiveness when technical indicators are used as input features, especially in comparison across multiple model families and under realistic trading evaluation criteria.

This study seeks to address the problem by evaluating machine learning models for stock forecasting using technical indicators as engineered inputs, assessing their predictive capability, interpretability, and practical applicability.

1.9 Research Objectives

The main objective of this thesis is to evaluate the performance of machine learning models applied to stock technical analysis and determine their suitability for practical forecasting and decision-making.

This objective expands into the following sub-objectives:

1. To collect, preprocess, and structure historical stock data suitable for machine learning and deep learning models.
2. To compute and integrate a diverse set of technical indicators as engineered feature representations.
3. To train and evaluate multiple machine learning algorithms, including classical ML models and neural architectures.
4. To compare model performance using both statistical accuracy metrics and financially meaningful performance metrics.
5. To interpret and analyze model behavior to assess generalization capability and practical feasibility in real-world markets.

1.10 Research Questions

Aligned with the objectives, the study aims to answer the following questions:

1. Do machine learning models improve prediction performance when combined with technical analysis indicators compared to raw prices?
2. Which category of model classical machine learning, deep learning, or hybrid performs best under consistent evaluation?
3. Can models trained on historical pricing patterns generalize to unseen market data, particularly in volatile environments?
4. Do statistical accuracy metrics correlate reliably with trading performance metrics such as Sharpe ratio or return on investment (ROI)?
5. Are models based on technical analysis suitable for deployment in emerging markets such as India, or do market conditions reduce their effectiveness?

Answering these questions provides measurable insight into the viability and limitations of applying machine learning to technical stock forecasting.

1.11 Scope, Limitations, and Assumptions

The scope of this research is restricted to:

1. Historical stock price data
2. Daily time-series resolution
3. Models trained on select high-volume stocks
4. Forecast horizon limited to short-term directional or price movement prediction

Key limitations include:

1. Results may vary across different stocks, time horizons, and economic conditions.
2. The study excludes macroeconomic indicators, textual sentiment, or high-frequency tick data.
3. Interpretability challenges may persist for deeper neural architectures.
4. Model performance may degrade during extreme market volatility or structural regime shifts.
5. Assumptions include the availability of consistent market data, stable economic conditions during evaluation, and reproducibility of indicator formulas across software implementations.

1.12 Novelty and Contributions

1. The contributions of this research are summarized as follows:
2. A structured comparative study of classical ML, deep learning, and hybrid models using a uniform technical analysis-based dataset.
3. Inclusion of both accuracy metrics and financial performance metrics, aligning evaluation with real-world trading relevance.
4. Application of machine learning-driven technical analysis specifically to Indian market conditions, addressing a documented research gap.
5. Development of a replicable preprocessing and modeling pipeline for future applied research and industry use.

These contributions offer practical value for quantitative researchers, retail investors, academic researchers, and financial institutions.

Chapter 2

Literature Review

2.1 Stock Market Forecasting and Technical Analysis

2.1.1 Introduction

Before the advent of machines and computers for visualization the technical analysis of stocks was limited to rule based trading systems derived from classical economic forecasting. Later with the rise of computer age and availability of data visualization trades started to rely on visual interpretation of price patterns and trends. Signals and moving averages to make investment decisions.

Brock, Lakonishok, and LeBaron (1992) [1], decisions provided the first systematic evaluation of technical analysis that demonstrated that rules such as Moving Average crossovers had statistical significance and could give significant returns in right market conditions. Rule Based forecasting [2]. is however limited by chaotic, non linear and non stationary nature of Stock markets. As time and technology progressed the research in technical analysis moved from rule based systems to more computational modelling to improve predictive stability and reduce human error and bias. This also increased the scope of Technical Analysis by including different indicators for volatility, momentum and behavioural signals. In the following Literature Review I will try to show the gradual shift from traditional methods the modern techniques using machine learning and deep learning for market forecasting. [3] [4]

2.1.2 Classical Forecasting Techniques

Older or classical Forecasting of time series data relied on deterministic and statistical framework such as Simple Moving Average, or Auto Regressive Integrated Moving

Average (ARIMA). Moving averages. They served as the baseline tool for detecting a trend and smoothing it for better understanding and visualization. But these indicators are limited by lag and inability to adjust to extreme volatility. [1]

ARIMA, introduced by Box and Jenkins (1970) [5], is a systematic approach to time series modelling that takes into consideration autoregression, differencing and noise smoothing. ARIMA assumes stationarity and linear structure conditions, which is rarely the case with stock markets. [6] There were improvements to ARIMA such as Seasonal ARIMA and volatility aware extension like GARCH that further improved the predictions that could be made but still struggled to accurately capture the market movement. [7]

In the case of Long term horizon forecasting Holt Winter method was significantly more successful in modelling trends and seasonality. Still it had its limitations during rapid spikes in trends, or reversals, or significant movement driven spikes.[8]. These limitations lead to shift towards approaches that could capture nonlinear and adaptive Realtime dynamic dependencies.

2.1.3 Traditional Trading Strategies

Research by Murphy(1999) [9] and Achelis (2001) [10] brought indicators taxonomies and classified them into Trend Following like Simple Moving Average (SMA), Exponential Moving Average (EMA), Moving Average Convergence Divergence (MACD), Momentum based (RSI, Stochastic Oscillators, Williams %R), Volatility based (ART, Bollinger Bands), and volume based (OBV) categories.

Brock et al. (1992) [1], shows that moving average breakout and support resistance rules generate abnormal returns when compared with random walk baseline. Similarly Neely et al. (2014) [11], has shown that RSI and MACD exhibit validity in market and repeatedly out performed as compared to random walk hypothesis assumptions [3].

Park and Irwin's meta-study (2007) [12] established that multi-indicators system could consistently outperform any single rule strategy. This was especially true in volatile environments.

2.1.4 Traditional Machine Learning models in Forecasting

Traditional machine learning techniques started to be used for forecasting with the advent of computer age and availability of powerful hardware to train these models. Early machine learning models addressed the issue of bridging the gap between rule based trading and structural statistical forecasting.

Support Vector Machine (SVM), Random Forest (RF), Decision Tree (DT) and Naïve Bayes formed the foundation of the machine learning models during this time. SVM was used in directional prediction and showed good results when trained with technical indicators. Combining Bollinger Band, moving average, stochastic Oscillator, RSI and Aroon Oscillator gave an accuracy of 77.8% on volatile dataset. [13]. Krenel-based SVM are still used frequently due to their ability to model non linear boundaries, even with smaller datasets.

Similarly Decision tree based models were traditionally used to due to their interpretability that is very important in stock market as money is involved. Studies have also applied multiple indicators to NSE equities reporting an accuracy upwards of 80%, showing that simpler models can outperform complex models when used with indicators in certain environments. [14].

2.2 AI Driven Stock Forecasting

As the technology and especially the advent of extremely good computers with very high computational ability, there was use of Machine learning in every field. Stock forecasting was no different with infact one of the most profit driven use of Machine learning. And hence a lot of bit financial institues became interested in the use of ML. They worked on diverse datasets and including as many features and aspects of the financial space.

Machine learning studies such as Huang et al. [1] demonstrated that supervised learning techniques could capture price dynamics more effectively than rule-based or traditional indicators based models. Even so they were often limited by feature dependence and also poor generalization. They are also extremely sensitive to non-stationary behavior.

Introduction of RNN was already a huge leap forward in creating model that need to consider temporal data. Hochreiter and Schmidhuber's foundational Long Short-Term Memory model [2] made it possible to retain long-term dependencies in sequential data, and also addressed the vanishing gradient limitations that is very common in RNNs.

Cho et al. [3] introduced the Gated Recurrent Unit (GRU), created a simplified gating mechanism while retaining temporal memory properties. GRUs are extremely efficient and hence became immediate favourite in financial forecasting. It also helps that they can generalize and converge well with limited amount of data as compared to LSTM.

This trend continues with Smarte and Ho [4] and Zhu [5], who showed that RNNs

remain one of the core part of financial forecasting and stock analysis.

Convolutional Neural Networks (CNNs), introduced by LeCun et al. [6], used for feature extraction expectailly in structure domain.Jin [7] included CNNs in the stock forecasting and stock data analysis.

Vaswani et al. [8] added Transformers to the mix of algorithms used in training forecasting models. The success of transformer in the natural language processing meant that they could be included in sentimental analysis for predicting stock prices based on the overall market temperament based on news as well as social media [9], multimodal forecasting [10], and LLM-driven trading pipelines [11]. Cilingiroglu [12] used transformers to create models for inter industrial patter recognition.

2.3 Comparison Between different Model and re-search

Table 2.1: Summary of Existing Machine Learning, Deep Learning, and Hybrid Approaches in Stock Market Forecasting

Study / Reference	Technical Indicators Used	Methodology and Main Findings
Huang et al., 2022 [1]	Standard price-based indicators and derived historical features	Compared classical machine learning models for global stock markets. Found that ML models outperform traditional statistical forecasting but struggle during high volatility.
Jin, 2024 [7]	CNN-derived feature maps + classical stock indicators	Proposed a hybrid CNN-based deep learning model to extract local patterns from technical indicators. Demonstrated improved performance over shallow networks.
Smarte & Ho, 2021 [4]	Broad survey of technical indicators including RSI, MACD, EMA, Bollinger Bands	Concluded that deep learning models outperform SVM/Random Forest when time dependency and financial noise are considered.
Khanpuri et al., 2024 [9]	Indicators integrated with textual sentiment signals	Combined sentiment from fundamental data with machine learning classifiers. Demonstrated that hybrid sentiment + indicator models outperform indicator-only systems.

Study / Reference	Technical Indicators Used	Methodology and Main Findings
Huang et al., 2025 [10]	Mixed quantitative indicators + deep learning feature extractions	Surveyed the evolution of modern quantitative investment from LSTM to LLM-based pipelines. Highlighted multimodal architectures as emerging state-of-the-art.
Cao et al., 2024 [11]	No traditional indicators; used contextual embeddings from market text	Applied LLMs for market forecasting. Demonstrated that language-driven forecasting outperforms numerical-only models under news-driven volatility.
Balasubramanian et al., 2024 [13]	Experimental biometric signals alongside price indicators	Showed that non-price features such as emotional states may correlate with market responses, suggesting behavioral-finance-based augmentation.
Mokhtari et al., 2021 [14]	Price-based indicators with reinforcement learning signals	Discussed the role of AI in automated trading. Concluded that reinforcement learning enhances long-term portfolio performance over static ML predictors.
Zhu, 2025 [5]	Survey of indicator-based forecasting strategies	Highlighted that technical-analysis-driven ML systems remain effective when paired with deep architectures such as LSTM or transformers.

2.4 Key Findings, Strengths, and Limitations

Table 2.2: Comparison of machine learning, hybrid deep learning, and emerging AI approaches for stock market prediction.

Study / Approach	Technical Indicators Used	Algorithm / Model	Key Results / Findings
Huang et al., 2022 [1]	Historical price-derived indicators (general TA-based dataset)	Classical ML models (SVM, RF, Logistic Regression)	Machine learning models outperform statistical baselines but suffer from instability under volatile market regimes.
Jin, 2024 [7]	CNN-generated feature maps + TA-based indicator inputs	Hybrid CNN-based deep learning system	CNN-based feature extraction improves modeling of local temporal patterns and yields better accuracy than traditional shallow NN architectures.
Smarte & Ho, 2021 [4]	RSI, MACD, EMA, Bollinger Bands, ADX (survey dataset)	Comparative ML/DL review	Concluded that deep learning (LSTM/GRU) consistently outperforms SVM/Decision Trees when sequential dependencies and market noise are considered.
Khanpuri et al., 2024 [9]	Technical indicators + fundamental textual sentiment	Sentiment-based ML pipeline	Hybrid indicator + sentiment models outperform indicator-only models when predicting post-news volatility and direction.
Mokhtari et al., 2021 [14]	Historical price and technical-analysis signals	Reinforcement Learning (A2C, DDPG)	RL-based systems outperform static ML models in long-term trading environments; demonstrated positive risk-adjusted returns relative to baseline benchmarks.

Study / Approach	Technical Indicators Used	Algorithm / Model	Key Results / Findings
Cao et al., 2024 [11]	No indicators; text-based embeddings extracted from news and reports	LLM-based forecasting	LLM models respond well to news-driven volatility and show stronger adaptability compared to purely numerical models.
Huang et al., 2025 [10]	Mixed quantitative TA indicators + deep embeddings	Survey of DL → LLM evolution	Identifies transformer and LLM-based frameworks as the next major stage in quantitative finance research.
Zhu, 2025 [5]	Mixed indicator-based forecast scenarios	Review of AI models for forecasting	Reinforces the need for hybrid pipelines; concludes combined sentiment + price features outperform single-signal systems.
Gupta, 2024 [15]	Price-driven TA signals + strategy metadata	AI-assisted quantitative trading	Demonstrated improved algorithmic trading performance when AI was integrated into execution and portfolio optimization strategies.

2.5 Evaluation Challenges, Research Gaps and Inconsistencies

- Data consistency and ways to deal with data inconsistencies when it comes to stock data
- Overfitting models, to particular segments based on number of stocks available.
- Limited generalizability overall to market properties.
- Clarity and explainability of models
- Indian stock exchanges remain unexplored for forecasting behaviour.
- Inconsistency in market trends caused due to India specific events such as government policy, overall impact of imports, and depends on other markets.
- Different Indian industries are dependent on different factors that set their price trends. Capturing them is difficult as they are not consistent with each other.
- Different industries in india is also dependent on different geo political regions and thus unstability in these regions causes different impact on the trends experienced by the overall market.
- Service industries remain one of the biggest contributors in the Indian stock market and have their own challenges.

Chapter 3

Methodology

3.1 Overview

The chapter describes the framework used to build and train the models and then their use in the project. This following sections each describe the preprocessing methodology, the feature engineering used, the training procedures, precision as well as the front and back end of the software developed to use the models to make it more user friendly. The model training include training with both technical indicators and raw data. The following sections also describe the models training done with single company data and also with multi company data. I have also discussed the models architecture.

3.2 System Workflow

The following diagram shows an overview of the complete system. In the following section I will go in detail about each part of the system.

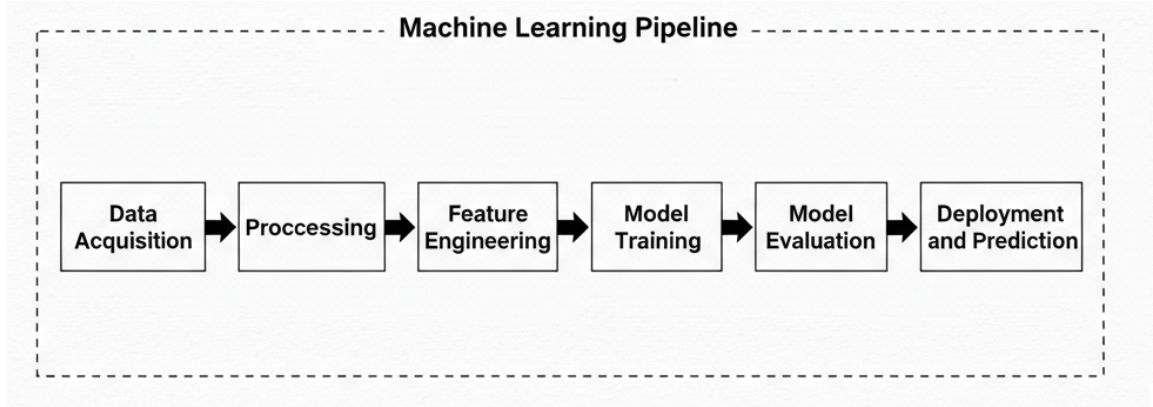


Figure 3.1: System Flow

3.3 Data Acquisition

While researching data acquisition there were several options that I had explored. Which included the free option like yahoo finance and other paid options. But finally I decided to go with Kite Connect API provided by Zerodha.

Table 3.1: Dataset Categorization Based on Usage and Scope

Dataset Type	Usage	Scope
Company-specific dataset	LSTM & GRU individual models	Single company, 2-year window
NIFTY-based dataset	CNN-LSTM and baseline analysis	NIFTY 50 and NIFTY 200 constituents
Full-market dataset	Transformer-based architecture and cross-learning	All available NSE-listed companies

Kite Connect API provided the best options when it came to the number of requests, daily limits, overall availability of data and most importantly cost. Yahoo Finance, being free, still has questionable legal circumstances as yahoo does not officially support using the api in the way and is based on web scrapping for its data. Kite Connect API on the other hand is completely supported by the company thus making it reliable.

Kite Connect API provides Open, Close, High, Low, and Volume data in the JSON format for all companies listed on NSE and BSE stock exchanges.

As Covid had impacted the markets in a major way, seeing a steep decline in price and then an unnatural rise in price as the market recovered, in this project to maintain consistency and I have considered data from the last two years. It captures the natural movement of market and thus helps to capture the patterns better than if covid period data was considered.

While sampling data for training, each day's data is considered, and period of 20

days is treated as a set. Saturday, Sunday and other Holidays when the stock market was closed for trading are skipped as they do not impact the data.

3.4 Data Preprocessing

Preprocessing data is an important step in training a model as it determines the accuracy of the trained model. The data should be consistent without outliers as they could skew the overall results. Similarly missing data should also be handled.

3.4.1 Handling Missing Values

Holidays when stock markets remain closed do not produce any data and is automatically handled by the Kite Connect API as it skips those days from the data it sends. As while preparing data the dates are only considered to maintain the series, it does not need to be handled separately.

Missing Data issue is mostly encountered for smaller companies that do not consistently train and can have skewed data for the days when there is huge movement seen in the stocks. These stocks are completely skipped during training for the models trained on All stock data.

Handling Non-Trading Days

Market does not operate on Saturday and Sundays as well as on public holidays. Thus we have missing data for those days.

Forward feeding could be used in this case as that does not effect or impact the temporal properties and pattern of the data.

$$x_t = \begin{cases} x_t, & \text{if value exists} \\ x_{t-1}, & \text{otherwise} \end{cases} \quad (3.1)$$

Missing Volume Replacement

There were rare instance where the volume data was missing. Volume is one of the most important feature in the training the model to understand the market dynamics as it reflects market participation and thus liquidity. Removing or replacing the value with forward filling could impact overall behaviour that was suppose to be learnt by the model.

The missing volume values were replaced with 7 days rolling average that takes into consideration a larger span of time than just forward filling.

$$Volume_t = \frac{1}{7} \sum_{i=1}^7 Volume_{t-i} \quad (3.2)$$

This method I found was much better than removing data, or replacing with complete data average or repeating prior values, in preserving the overall context. Thus preventing sudden change that could be introduced by other methods, but also preserves the overall movement that is expected for that time.

Missing Indicator Rows

Some Indicators are calculated considering the past few days data such as Moving Average, or Volatility estimators. Thus they could need a minimum amount of past data to calculate for the set window. In Most cases the values are interpolated or could be entirely skipped.

$$D_{clean} = \{x \in D \mid \text{all features at } t \text{ are non-null}\} \quad (3.3)$$

Rational for Missing Data Strategy

Using different approaches for different data like missing base features, skipping companies with extreme volatility, rolling average for volume data ensures that the essence of the market behaviour can be learned by the model.

Following were the points considered while selecting the techniques used to overcome missing data.

- Help preserve historical trends
- Prevent artificially creating volatility or distortion in data.
- As using Time series model, ensure that the sequential nature of the data is maintained.

While training any machine learning model, and especially for time series model where sequential data is used there is a huge impact of data preprocessing on model accuracy and training convergence.

3.4.2 Scaling and Normalization

While training with multiple it is important that the data is normalised as the price can vary hugely between companies. The goal of training a time series model on for stock is to capture the different pattern and use this pattern to predict the next days closing. Normalisation ensures that the specific companies price are not considered, just the movement of price from day to day is considered.

$$x_{\text{scaled}} = \frac{x - \min(x)}{\max(x) - \min(x)} \quad (3.4)$$

During inference the min max of the that companies data is considered. Thus once the normalised result are inferred, that min max can then be used to calculate the scaled closing value.

Simple Min Max scaling is used to Normalise data in this project. The following formula is used for the time dependant features.

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (3.5)$$

3.5 Feature Engineering

I in this project have tried to compare the effectiveness of Feature Engineering vs the Raw Stock data. Thus feature engineering is a major part of this project.

Raw stock data considers valuable information, it can be prominently see in candle stick data, which has been used since a long time and is one of the foundational Technical analysis tools used. But for a machine learning model, especially those considering time series data, it could be helpful to consider other technical analysis indicators which could enhance the existing patterns in the data.

In this project I have tried to consider the combinations of Technical indicators that consider historical pricing and volume together so both are given equal importance. Technical Indicators transform raw values into structured input that enhance the pattern learning and thus improve the predictive performance of the model.

3.5.1 Raw Feature Selection

The raw data retrieved from Kite Connect API for each company contains the Open, High, Low, Close and Volume information for each day. These form the base features of the stock data that is used to train the models.

The feature set is defined as:

$$X_t = \{Open_t, High_t, Low_t, Close_t, Volume_t\} \quad (3.6)$$

The consistency of base features between companies provides a solid base on which additional features can be calculated.

3.5.2 Windowing Strategy

As all the models trained in this project are time series models such as LSTM, GRU, Transformers and CNN-LSTM, it is important to maintain consistency between the windowing strategy that will work for all models so the results can be effectively compared.

$$X = [X_{t-n}, X_{t-n+1}, \dots, X_{t-1}] \rightarrow y = Close_t \quad (3.7)$$

$$\begin{aligned} X &= [\text{features for days } t - 19 \text{ to } t] \\ y &= \text{closing price at day } t + 1 \end{aligned}$$

Sliding window strategy is chosen for dividing data into sets. The window size of 20 is used throughout the project to maintain consistency and also maintain the scope of the project. I found that this window size was large enough that the model could observe short term trends and volatility behaviour but is not computationally as intensive due to the limitation of my hardware used in this project.

3.5.3 Training Test Validation Structuring

Dividing data into Training and Test is an important final step in feature engineering. It is important that the training as well as test data maintains the temporal order. Unlike other machine learning model training and testing where the data might be shuffled, time series model require that the data preserves its order both while it is used for training and then while testing.

Data is divided as follows :

$$D = D_{\text{train}} \cup D_{\text{test}}, \quad \text{with } t_{\text{train}} < t_{\text{test}} \quad (3.8)$$

Validation method uses time series split without any shuffling.

The callback method is used for early stopping and learning rate reduction. After

6 epochs of training without improvement in validation error, the training is stopped early so compute could be saved and training time could be reduced.

Loss curve and predictions are generated after training using training data set. This is an important visual representation in comparing the models.

3.5.4 Indices Used

Stock indices from NSE and BSE have been used as solid reflection of the overall health of market. Segment indices as well as the top stock are maintained by both BSE and NSE. In this project I have used the indices from NSE like NIFTY50, NIFTY100 and NIFTY200 as they give the best combination of the stable bigger companies of India in NIFTY50 and bit more diverse companies in NIFTY200.

3.6 Technical Indicators

Technical Indicators are mathematical transformation of raw features of historical price and volume data that are used to enhance and extract meaningful market patterns and thus help models learn the patterns better. Raw data can be combined to reveal information such as volatility, momentum, or even market sentiment which is not always directly visible or interpretable from raw features.

In this project I have tried to use the most widely used Technical Indicators that will help capture the trend, momentum, volatility and volume. The following sections I will discuss in detail the Technical indicators used and the purpose they serve in combination with other indicators. I will also discuss the advantages and disadvantages and if there are ways to mitigate the limitations while using them.

3.6.1 Exponential Moving Average (EMA)

The Exponential moving average is a derivative of Simple moving average that gives more weight to recent price values, thus making it more responsive to recent movement. Both Simple Moving Average and Exponential Moving average fall in the same category that they are used as trend indicators. Thus giving additional weight-age to recent data EMA reacts better and follows trends more closely than SMA.

Exponential Moving Average is defined by the following formula.

$$EMA_t = \alpha \cdot P_t + (1 - \alpha) \cdot EMA_{t-1} \quad (3.9)$$

where:

- P_t is the closing price at time t
- $\alpha = \frac{2}{n+1}$ is the smoothing factor
- n is the lookback period (12 or 26 in this study)

Advantages:

- Main advantage of this indicator is that it reacts faster to the trend changes than Simple moving average.
- It helps indicate the market direction and momentum.
- It can be used to enhance trends while being used for training.

Limitations:

- As it reacts to recent changes more than SMA, it might introduce noise during high market volatility.
- Is very market dependent and needs to have the window to be decided accordingly

Role in This Project:

In this project I have choosen EMA as the trend indicator. It helps smooth out the directional patterns ompared to just raw price squence that could vary wildly on some days. This is highly effective when considering large datasets.

The soothing of these irregularities also help with better learning of the model as it complements how models like LSTM and CNN-LSTM work underneath.

3.6.2 Relative Strength Index (RSI)

The Relative Strength Index (RSI) is traditioanlly used to understand if a stock is overbought or oversold.

RSI indicates the price momentum, meaning the speed at which the price changes. This can be an effective indicator of petential buying or selling signal.

The RSI is calculated as:

$$RS = \frac{\text{Average Gain}}{\text{Average Loss}} \quad (3.10)$$

$$RSI = 100 - \left(\frac{100}{1 + RS} \right) \quad (3.11)$$

Advantages:

- Captures short-term momentum
- Indicate reversal behavior
- Detecting buying or selling pressure

Limitations:

- They can be susceptible to false indicators especially in trend driven markets.
- Extremely sensitive to short term volatility.

Role in This Project:

RSI are used as a counterbalance to the trend heavy indicators such as EMA. It bring oscillatory behavior to the overall learning.

It is very effective when considering larger data sets, but can fail especially in saller data sets such as when considering single company data. In those small data sets the RAW features already contain enough momentum information.

3.6.3 Average True Range (ATR)

The Average True Range (ATR) is a volatility indicator.

It is designed to measure the degree of price range expansion or contraction over time.

$$TR_t = \max \left(\begin{array}{l} H_t - L_t, \\ |H_t - C_{t-1}|, \\ |L_t - C_{t-1}| \end{array} \right) \quad (3.12)$$

$$ATR_t = EMA(TR_t, 14) \quad (3.13)$$

Advantages:

- Quantify market volatility

Limitations:

- Price direction is not represented.
- Could emplify noise, especially in smaller dataset.

Role in This Project:

ATR is used to indicate volatility dynamic to the feature set. The indicator was especially useful when used with larger datasets. It was detrimental to the training when used with smaller dataset as it could introducte a lot of noise.

3.6.4 On-Balance Volume (OBV)

On-Balance Volume (OBV) is a volume-based indicator that can be used to track buying and selling pressure.

$$OBV_t = \begin{cases} OBV_{t-1} + V_t, & \text{if } C_t > C_{t-1} \\ OBV_{t-1} - V_t, & \text{if } C_t < C_{t-1} \\ OBV_{t-1}, & \text{otherwise} \end{cases} \quad (3.14)$$

V_t - traded volume.

Advantages:

- Volume shifts preceding price movement
- Helps reveal accumulation or distribution patterns

Limitations:

- Is very Sensitive to volume spikes
- Does not work well with low liquidity stocks

3.6.5 Summary of Indicator Contribution

Indicator based features are used in training, both in larger as well as smaller models. The single stock data do not benefit as much from the stock indicators, but multi company models trained with features that include the Indicators show significantly better results. This is because in larger datasets they enhance the existing patterns, that they represent.

3.7 Model Training and Validation Datasets

Train/Test split: 80% / 20%

Validation method: time-series split (no shuffling)

Callbacks: early stopping and learning rate reduction

3.8 Prediction and Deployment Pipeline

Following are the different steps and data flow in the Project.

Table 3.2: Dataset Splitting Strategy

Dataset Type	Split
Training Set	80% of historical sequence
Testing Set	Remaining 20%

3.9 Architecture diagram of data flow

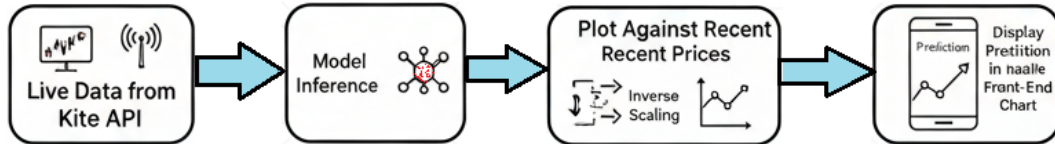


Figure 3.2: Architecture of Data Flow.

- Historical data for all symbols available on NSE was collected at once and then updated as the company symbol is searched in the interface.
- If kite connect API is not available then this historial data is used.
- While training bigger models it is important that all the data is available at once, as failure to retrieve data from API could fail the training.
- Every time a user searches for a symbol in the interface new data for that symbol till date is fetched and then used to train the models specific for that company and also used for inference with the larger models.
- Once live data is fetched, test data set is created with the last 90 days of data and is used to plot predictions against actual price. This graph gives a visual representation of the effective ness of that model for that specific companies stock data.
- Finally all the model predictions are calculated and graphs of test as well as the complete history of that company over the past 2 years is plotted.

3.10 Hyperparameter Configuration

3.11 Evaluation Metrics

Table 3.3: Model Hyperparameters

Parameter	Value / Range
Epochs	50–200 (early stopping applied)
Batch Size	32–64
Optimizer	Adam
Learning Rate	0.001 (adaptive reduction on plateau)
Loss Function	Mean Squared Error (MSE)

Table 3.4: Model Evaluation Metrics

Metric	Description
MAE	Measures average magnitude of error (Mean Absolute Error)
RMSE	Penalizes larger errors more heavily (Root Mean Squared Error)
MAPE	Expresses prediction error as a percentage (Mean Absolute Percentage Error)
R ² Score	Measures goodness of fit (Coefficient of Determination)

Chapter 4

Experimental Setup and Implementation

4.1 Overview

In this section I will try to give an overview of the complete experimental setup including both Software and hardware used to train, evaluate and compare the machine learning model that have been designed to give next day predictions.

4.2 Hardware Configuration

All experiments were performed on my personal PC. The following table contains the details of the machine used during the complete duration of the project.

Table 4.1: Recommended Hardware Specifications

Component	Specification
Processor	AMD Ryzen 5600G or equivalent multi-core CPU
RAM	16 GB DDR4
Storage	SSD (for faster dataset access and training runs)
GPU (optional)	AMD RX580 (Limited training Support)

The hardware specifications become important as the libraries used and thus the training time can be heavily impacted by the hardware being used. In my case as I was using AMD RX580 as the GPU which is not supported by tensorflow that is used in this project as library for training and most other popular libraries with good support, all the training was mostly completed on the CPU.

Even though most training could use CPU, GPU could speed up training time significantly in most cases by many times. This becomes especially significant when larger models, like transformer or CNN LSTM models trained on all companies data, it took significant time upwards of hours to complete. Where as if GPU acceleration were used with the CUDA cores of Nvidia, it could have been significantly reduced.

4.3 Software Stack

All the experiments and training was completed on a Windows machine. The complete project uses Tensorflow as the main library/framework used for building models. Tensorflow was chosen due to its industrial importance and robustness as well as excellent support available online, making it a proven framework used for machine learning. Scikit-learn is used for quick experimentation due to its ease of use and great support for academic processes.

Following table gives a more complete list of software being used.

Table 4.2: Project Technology Stack

Category	Libraries / Tools
Programming Language	Python
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn, Plotly
Machine Learning	scikit-learn
Deep Learning	TensorFlow / Keras
API Data Access	Kite Connect API
Deployment	Custom front-end interface for visualization

4.4 Kite Connect API

Zerodha provides the Kite Connect API as a tool for developer to build custom trading applications. It offers real time order execution and portfolio management. But more important to this project, they provide live stock market data streaming for equities, commodities and MF's.

Their api can be implemented directly as HTTP API requests with any language of choice but in this case I have used the opensource API helper that is provided by Zerodha for python that helps directly pull data from their APIs.

Listing 4.1: Installing KiteConnect

```
pip install kiteconnect
```

The complete documntation for this could be found at

Listing 4.2: KiteConnect GitHub Repository URL

`https://github.com/zerodha/pykiteconnect`

4.4.1 Developer Account

A developer account needs to be setup for a user to access the API.

The developer account lets users create an app in the zerodha kite connect API dashboard that will help the user get a api key and api secret that is used to authenticate users and then authorize any changes.

- Create Developer Account
- App Creation
- Set Redirect URL
- Get API Keys

4.5 Authentication process

In this program that I have created I have used a python script that help go through the authentication process which will generate a request token.

The request token is used to authorize any changes to the users account and his portfolio.

Running ***python scripts/generate.access.token.py*** to generate Access Token that needs to be refereshed everyday. This is then stored in credentials file in the root of the project and can be used for further requests.

It is important to make sure that this file is not shared online and does not get included in the github repo so should be excluded as it contains sensitive information about the kite connect account that could to make unwanted transactions.

Listing 4.3: API Credentials Configuration

```
{
  "API_KEY": "",
  "API_SECRET": "",
  "ACCESS_TOKEN": "",
  "REQUEST_TOKEN": ""
}
```

4.6 Getting Historical Data

I used a script that would get each companies stock historical data and store in the data folder so that it could be used later in any combination to train models as needed.

Following are the different configuration of parameter that could be used with the Kite Connect API

4.6.1 Historical Data Endpoint

Method GET

Endpoint /instruments/historical/{:instrument_token}/{:interval}

Description Retrieve historical candle records for a specified financial instrument.

4.6.2 Path Parameters

:instrument_token Unique identifier of the financial instrument for which historical data is requested. This value is obtained using the `instrument list` endpoint.

:interval Specifies the time interval of each OHLCV candle record. Supported values include:

- minute
- day
- 3minute
- 5minute
- 10minute
- 15minute
- 30minute
- 60minute

4.6.3 Query Parameters

from Defines the start of the historical window using the format `yyyy-mm-dd hh:mm:ss`.

to Defines the end of the historical window using the format `yyyy-mm-dd hh:mm:ss`.

continuous Accepts **0** or **1**. Set to **1** to retrieve continuous historical data streams.

oi Accepts **0** or **1**. When set to **1**, the response includes **Open Interest (OI)** data.

4.6.4 Stock Data Example

Listing 4.4: KiteConnect Candle Data Response

```
{
  "status": "success",
  "data": {
    "candles": [
      [
        "2019-12-04T09:15:00+0530",
        12009.9,
        12019.35,
        12001.25,
        12001.5,
        163275,
        13667775
      ],
      [
        "2019-12-04T09:16:00+0530",
        12001,
        12003,
        11998.25,
        12001,
        105750,
        13667775
      ],
      [
        "2019-12-04T09:17:00+0530",
        12001,
        12001,
        11995.1,
        11998.55,
        48450,
        13758000
      ],
      [
```



```

        "2019-12-04T09:18:00+0530",
        11997.8,
        12002,
        11996.25,
        12001.55,
        52875,
        13758000
    ],
    [
        "2019-12-04T09:19:00+0530",
        12002.35,
        12007,
        12001.45,
        12007,
        52200,
        13758000
    ],
    [
        "2019-12-04T09:20:00+0530",
        12006.95,
        12009.25,
        11999.6,
        11999.6,
        65325,
        13777050
    ]
]
}
}

```

Chapter 5

Model Details

5.1 Gate Recurrent Unit (GRU) Models

5.1.1 Introduction to GRU Architecture

The Gated Recurrent Unit is a type of Recurrent Neural Network that is designed to process time series sequential data by retaining temporal patterns during training. While neural networks can produce impressive results by learning patterns, time series models such as GRU are specifically designed to consider sequential data, and maintain memory of the past that will then influence the future training.

GRU uses gating mechanism to determine how much historical information should be retained and what can be forgotten during the sequence processing. The following diagram shows the internal architecture of GRU.

There are two main or primary gates of GRU :

- Update Gate – It controls how much of the past information is retained so it remains relevant for the next output.
- Reset Gate – It determines how much of the previous memory should be ignored while it integrates the new input signal.

This gated mechanism helps GRU avoid the common problem of vanishing gradient and exploding gradient commonly seen in the standard RNN and thus allows GRU to learn long term dependencies efficiently.

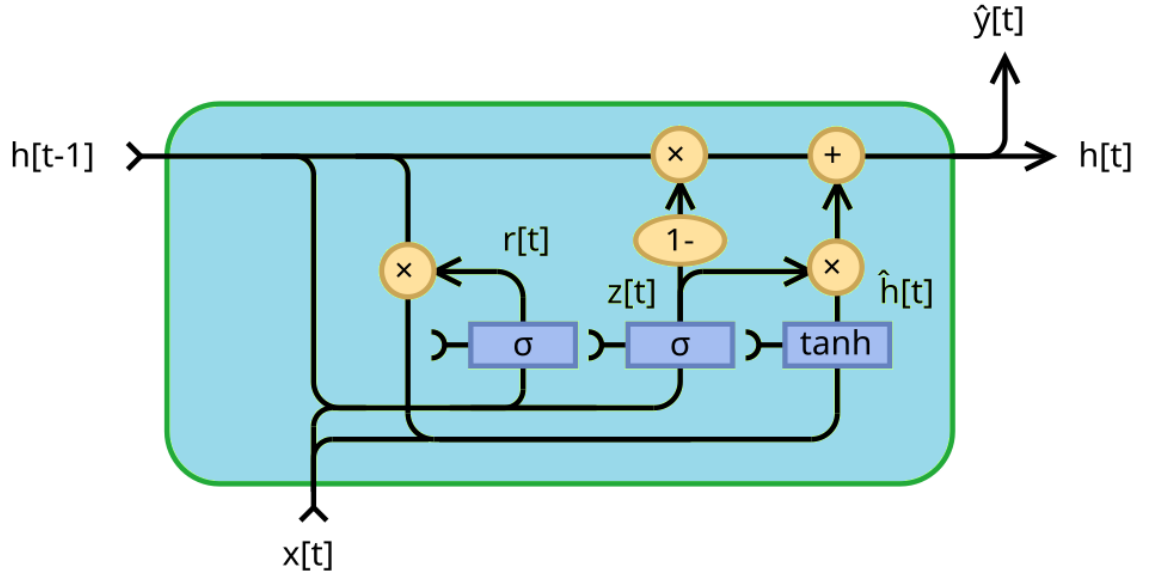


Figure 5.1: Architecture of the GRU block.

5.1.2 Mathematical Model for GRU

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

Reset Gate

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

Update Gate

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

Candidate Hidden State

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Final Hidden State

Symbol	Description
x_t	Input vector at time t .
h_{t-1}	Hidden state from the previous time step $t - 1$.
h_t	Final hidden state (output) at time t .
W_i, U_i, b_i	Weight matrices (W for input, U for recurrent state) and bias vectors to be learned.
$\sigma(\cdot)$	The sigmoid activation function ($\frac{1}{1+e^{-x}}$), which outputs values between 0 and 1.
$\tanh(\cdot)$	The hyperbolic tangent activation function , which outputs values between -1 and 1.
$\odot$	The Hadamard product (element-wise multiplication).

Table 5.1: Key Notations for Gated Recurrent Unit (GRU) Equations

5.1.3 Advantages and Disadvantages of GRUs Category Details

Category	Details
Advantages	• Faster training compared to LSTM due to fewer parameters • Performs well with limited data • Avoids vanishing gradient problem better than simple RNNs • Works effectively without feature engineering in many cases
Disadvantages	• Slightly less expressive than LSTMs for complex long-range patterns • May overfit when trained only on one stock without regularization • Interpretability remains limited compared to simpler models
Shortcomings in Context	• Requires careful hyperparameter tuning to avoid memorizing past trends • Sensitivity to scaling and outlier behavior in noisy financial datasets

Table 5.2: Assessment of Model Characteristics (likely GRU/Simple RNN variant)

5.1.4 Use of GRU in Project

I wanted a baseline model that was better than RNNs but was simple, fast to train and could give good results with limited data available, such as training a model from a single companies data. GRU fulfils this role in spades. It generalises well on smaller dataset, but also learns patterns of that specific smaller dataset making it extremely capable. It also converges faster making training time smaller especially

for the real-time nature of training that is required for the application. GRU gives perfect balance between model size, complexity and generalisation ability.

GRU also performs particularly well when trained with just the Raw features of stock data, Open High Low Close and Volume. Models trained as such produce stable and accurate predictions for that particular dataset. This also makes the model highly specific to that stock data, sacrificing generalisation for pattern learning and accuracy for that specific stock.

When GRU is trained on broader dataset such as NIFTY50 OR NIFTY200 the model definitely benefits from the broader dataset and variability, but it still lacks behind in performance as compared to models trained with LSTM, as LSTM is able to capture patterns better as it retains more of the past effecting the overall training.

5.1.5 GRU Model training Configuration

Model 1 - Only Raw Stock Features (No Indicators)

This is the model that is trained on just the raw stock data for a single company that is the lightest model used in this Project. This light structure produces faster convergence and strong performance when considered for a single stock data, making it a strong choice when the data is limited.

It consist of a single GRU layer, followed by dropout and dense layers to generate outputs. It is trained for 60 epochs with early stopping when the validation error stops improving.

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 64)	13,632
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Table 5.3: Model Architecture Summary

Model 2 – Technical Indicators included with Raw Stock Features

In the second configuration that I used in this project I trained GRU model with engineered features that capture trend, volatility, and volume.

To maintain consistency between models and make them comparable, the same Engineered Features are used as shown in the following table

Type	Indicator	Description
Trend	EMA (12, 26)	Exponential moving averages
Momentum	RSI (14)	Relative Strength Index
Volatility	ATR (14)	Average True Range
Volume-based	OBV	On-Balance Volume

Table 5.4: Summary of Technical Indicators

In this configuration, two GRU layers, each followed by a dropout layer is used, then followed by 2 dense layers to bring it all together. Similar to earlier model, it is also trained to 60 epochs, with early stopping if the validation error stops improving.

In most cases I found that even though the GRU model trained with indicators performed better when considering stock data from multiple companies, say using NIFTY50, but underperformed when it was trained on a single companies data. This could mostly be attributed to being not enough data and too many features to consider making the model over fit and unstable.

Layer (type)	Output Shape	Param #
gru_1 (GRU)	(None, 20, 128)	53,760
dropout_1 (Dropout)	(None, 20, 128)	0
gru_2 (GRU)	(None, 64)	37,248
dropout_2 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

Table 5.5: Stacked GRU Model Architecture Summary

5.1.6 Summary of GRU model Performance

Key takeaways from the GRU experimentation include:

- GRU offered the best performance and quickest convergence as compared with other time series models.
- It worked best when trained with minimum features when used with a smaller data set, as was done when using only a single companies stock data.
- When technical indicators were used it best generalised when larger data set say NIFTY50, NIFTY200 were used.
- GRU thus become the baseline model against which other models could be compared for stock market forecasting.

5.2 Long Short-Term Memory (LSTM) Model

5.2.1 Overview of LSTM Architecture

LSTM is considered one of the most popular extension of Recurrent Neural Networks (RNNs), which addresses the biggest issue with RNNs of vanishing and exploding gradient. It is designed to effectively learn patterns in time series data. Like GRU, LSTM achieves it through gated structures, specifically input gate, forget gate and output gate. These gated structures regulate how data flows forward through time and how past data is retained.

A new Memory state or Cell State is introduced in LSTM that stores useful past information across longer time span than RNNs and also GRU. This makes LSTM the choice for larger stock data, and training of models that generalise well and can learn historical patterns in financial time series forecasting. This makes them adaptable to short term gradual momentum shifts as well as long term historical patterns observed.

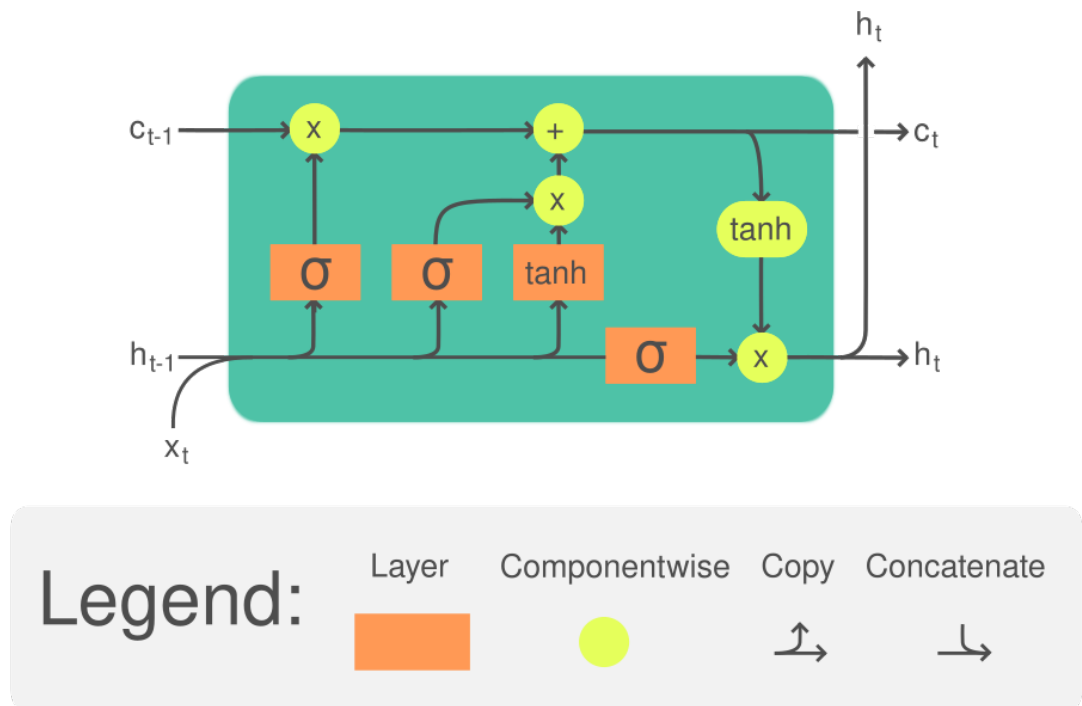


Figure 5.2: Architecture of the LSTM block.

5.2.2 Why Use LSTM in time series forecasting

LSTM networks are capable of learning temporal dependencies very well and are thus used in time series forecasting application such as in this case Stock Market Forecasting. They are capable of learning longer patterns in data such as trend formations, and reversal patterns as well as handle non linear input.

In stock forecasting it is often said, History repeats itself, and the whole concept of Stock Forecasting based on Technical Analysis is based on this principle. LSTM models specialise in retaining information such as trends, oscillations and volatility cycle over time and are therefore one of the best choice in predicting next days closing price when the model is trained with sufficiently large dataset.

5.2.3 Mathematical Model for LSTM

$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$	Forget Gate
$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$	Input Gate
$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$	Output Gate
$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$	Candidate Cell State
$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	Updated Cell State
$h_t = o_t \odot \tanh(c_t)$	Final Hidden State (Output)

Symbol	Description
x_t	Input vector at time step t . Typically contains the feature values (e.g., OHLCV and/or indicators) for the current day.
h_{t-1}	Hidden state from the previous time step $t - 1$. Encodes past information propagated through the sequence.
h_t	Final hidden state (output) at time step t , used either directly for prediction or passed to subsequent layers.
c_{t-1}	Previous cell state at time step $t - 1$, representing long-term memory.
c_t	Updated cell state at time step t , storing both retained and newly added information.
f_t	Forget gate vector at time t , controlling which information from c_{t-1} is retained ($f_t \approx 1$) or discarded ($f_t \approx 0$).
Continued on next page	

Symbol	Description
i_t	Input gate vector at time t , controlling how much of the candidate state $\tilde{c}_t$ is written into the cell state.
o_t	Output gate vector at time t , controlling how much of the internal cell state c_t is exposed to the hidden state h_t .
$\tilde{c}_t$	Candidate cell state at time t computed from current input and previous hidden state, before being filtered by the input gate.
W_f, W_i, W_o, W_c	Weight matrices applied to the input x_t for the forget, input, output, and candidate cell computations respectively.
U_f, U_i, U_o, U_c	Recurrent weight matrices applied to the previous hidden state h_{t-1} for the forget, input, output, and candidate cell computations.
b_f, b_i, b_o, b_c	Bias vectors associated with the forget, input, output, and candidate cell transformations.
$\sigma(\cdot)$	Sigmoid activation function, $\sigma(x) = \frac{1}{1+e^{-x}}$, squashing values into the range $(0, 1)$ and used for gating.
$\tanh(\cdot)$	Hyperbolic tangent activation function, mapping values to $(-1, 1)$ and used to create bounded candidate and exposed states.
$\odot$	Hadamard (element-wise) product between two vectors or matrices.

Table 5.6: Key notations and equations for the Long Short-Term Memory (LSTM) network

5.2.4 Advantages and Disadvantages of LSTM

5.2.5 Role of LSTM in this Project

LSTM models are included in this project as a larger model type than GRU that can retain more dense information and thus can generalise better, across different datasets and feature complexity. The LSTM models showed strong performance when trained with single company as well as multi company data, overall showing the most balanced behaviour.

When the LSTM model was trained with raw features (OHLCV), the LSTM model prediction smoothly followed the price movement. This is very similar to GRU model even though GRU performed slightly better when only single company data

Category	Details
Advantages	• Able to learn long-term dependencies more effectively than standard RNNs or GRUs • Performs well on noisy sequences where patterns unfold gradually • Stable training dynamics for multi-stock datasets • Less prone to catastrophic forgetting
Disadvantages	• Higher computational cost due to larger number of parameters • Longer training time compared to GRU • May overfit when data availability is limited unless regularization is applied
Project Context Limitations	• When trained on a single stock, LSTM models may learn stock-specific behavior too well, reducing cross-stock generalization • Performance benefit compared to GRU is noticeable mainly when the dataset size increases

Table 5.7: Assessment of LSTM Model Characteristics

was considered, and LSTM performed much better with larger datasets. This stems from the models capability of memorising the stocks internal movement patterns.

When trained with larger dataset such as NIFTY50, NIFTY200 and all companies data, and especially with additional engineered features, like technical indicators for trend and volatility the LSTM architecture allowed the model to generalise very well to diverse market conditions. This shows that the LSTM is better suited for multiphase temporal patterns that occur across companies.

5.2.6 Trained LSTM Model Configurations

LSTM Model Without Indicators

This LSTM model that is trained only on raw features (OHLCV) input. It consist of 2 LSTM layers each followed by dropout layers and then 2 dense layers to bring the result together.

It was trained to 60 epoch with early stopping when the validation error stops getting better waiting for 6 epoch before stopping.

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 20, 128)	68,608
dropout (Dropout)	(None, 20, 128)	0
lstm_1 (LSTM)	(None, 64)	49,408
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Table 5.8: Stacked LSTM Model Architecture Summary

LSTM Model With Indicators

The second LSTM model was trained with the technical indicators, engineered features that are consistent with those used in GRU model training.

This configuration improves the model's ability to generalise even better as the engineered features emphasise the patterns. This is especially apparent form performance achieved by the LSTM with engineered features trained on all companies data.

Feature	GRU Model	LSTM Model
Architecture	128→64 GRU	128→64 LSTM
Parameters	Fewer	Slightly more
Long-term memory	Moderate	Stronger
Training speed	Faster	Slower
Use case	Noisy short-term data	Trendy long-term sequences

Table 5.9: Comparison of GRU vs. LSTM Architectures

- The indicator-based LSTM model generalized better when trained on multiple companies.

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 20, 128)	71,168
dropout_3 (Dropout)	(None, 20, 128)	0
lstm_1 (LSTM)	(None, 64)	49,408
dropout_4 (Dropout)	(None, 64)	0
dense_4 (Dense)	(None, 32)	2,080
dense_5 (Dense)	(None, 1)	33

Table 5.10: Stacked LSTM Model Architecture Summary

- The non-indicator model remained highly effective for a single company, especially when trained on two-year datasets.
- This reinforces the idea that feature engineering provides benefit primarily when dataset diversity increases.

5.2.7 Performance Summary

LSTM models provide a balance between size of training data required, features that is can accommodate and training time, giving better generalisation and robustness as compared with GRU models. GRU models even though could be trained faster and on smaller dataset have the inherent flaw of not generalising as well, which by itself stems from smaller data set and thus fewer patterns to learn from.

LSTM thus forms the foundation benchmark for other more complicated models, like CNN-LSTM model that I have explored later in this project. LSTM definitely showed better performance than the more complex models like transformers and CNN LSTM.

5.3 Transformer Model

5.3.1 Transformer Architecture

Transformers were originally introduced in natural language processing for sequence to sequence learning tasks, where long range dependencies and contextual relationship capturing as essential. Transformer unlike traditional RNNs and their derivatives do not rely on sequential processing. They instead use self attention mechanism that allow them to evaluate the importance of different time steps relative to one another within the input sequence.

This learning behaviour of transformers allow them to weigh which portion of the historical sequence are most relevant to the prediction task at any given time. Transformers are also extremely computationally efficient as the attention is computed in parallel rather than iteratively. This makes them capable of scaling and modelling longer temporal patterns than RNNs or LSTM models that degrade the memory overtime.

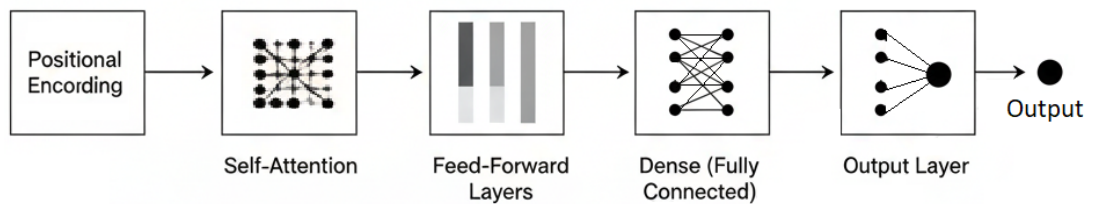


Figure 5.3: Architecture of the Transformer block.

5.3.2 Motivation for Using Transformer in this project

After trying out LSTM the next logical step was trying the training model with Transformer architecture. As transformer require larger data sets I did not try to train them on smaller dataset as they did not perform as well on them. They train better with larger data sets like all companies data and could be decent with NIFTY200.

- Capture long term dependencies
- No recursive backpropagation
- Process sequences in parallel
- Adapt dynamically to context
- Automatically prioritize relevant historical points

5.3.3 Mathematical Model for Transformer Encoder

Self-Attention Mechanism

$$\begin{aligned}
 Q &= XW_Q, \quad K = XW_K, \quad V = XW_V && \text{Linear Projections} \\
 A &= \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) && \text{Attention Weights} \\
 \text{SA}(X) &= AV && \text{Self-Attention Output}
 \end{aligned}$$

Multi-head Self Attention with H heads:

$$\begin{aligned}
 \text{head}_h &= \text{SA}(XW_Q^{(h)}, XW_K^{(h)}, XW_V^{(h)}) \\
 \text{MHA}(X) &= \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O
 \end{aligned}$$

Transformer Encoder Layer

$$\begin{aligned}
 H'_1 &= \text{LayerNorm}(X + \text{MHA}(X)) && \text{Residual + Norm after MHA} \\
 \text{FFN}(H'_1) &= \phi(H'_1W_1 + b_1)W_2 + b_2 && \text{Position-wise Feed-Forward} \\
 H_1 &= \text{LayerNorm}(H'_1 + \text{FFN}(H'_1)) && \text{Final Encoder Output for Layer}
 \end{aligned}$$

$\phi(\cdot)$ - ReLU or GELU activation.

Symbol	Description
X	Input sequence matrix of shape $T \times d_{\text{model}}$, where T is the sequence length (e.g., 20 days) and d_{model} is the embedding dimension. Includes feature embeddings and positional encodings.
Q, K, V	Query, Key, and Value matrices derived from X via learned linear projections. Used to compute attention scores and context.
W_Q, W_K, W_V	Projection weight matrices mapping the input X into query, key, and value spaces for single-head attention.
$W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$	Projection weight matrices for head h in multi-head attention. Each head learns its own subspace representation.
W_O	Output projection matrix applied to the concatenated multi-head outputs.
Continued on next page	

Symbol	Description
d_k	Dimensionality of the key (and query) vectors, used to scale the dot product in the attention mechanism.
A	Attention weight matrix obtained after applying the softmax function to the scaled dot products $QK^\top$. Each row represents a normalized attention distribution over time steps.
$\text{SA}(X)$	Single-head self-attention output, computed as AV , representing a weighted combination of value vectors.
head_h	Output of the h -th attention head. Each head focuses on different aspects or positions within the input sequence.
$\text{MHA}(X)$	Multi-head attention output formed by concatenating all heads and applying a final linear transformation.
H'_1	Intermediate hidden representation after adding the residual connection ($X + \text{MHA}(X)$) and applying layer normalization.
$\text{FFN}(\cdot)$	Position-wise feed-forward network applied independently to each time step, usually consisting of two linear layers with a nonlinearity.
W_1, W_2	Weight matrices of the feed-forward network, expanding and then projecting back the feature dimension.
b_1, b_2	Bias vectors associated with the feed-forward layers.
$\phi(\cdot)$	Nonlinear activation function applied in the FFN (e.g., ReLU or GELU).
H_1	Final output representation of the encoder layer after the second residual connection and layer normalization. This output is used either as input to the next encoder layer or for downstream prediction.
LayerNorm	Layer normalization function that normalizes activations across feature dimensions to stabilize and accelerate training.
Concat	Concatenation operation combining the outputs of multiple attention heads along the feature dimension.

Table 5.11: Key notations and equations for the Transformer encoder layer used in this project

Table 5.12: Analysis of a Deep Learning Model (Likely Transformer/Self-Attention-based) for Time Series Prediction

Category	Observations
Advantages	• Able to learn long-range and non-linear dependencies • Scales efficiently to large datasets • Attention mechanism helps prioritize meaningful time steps • Performs well when incorporating multiple feature types
Disadvantages	• Requires substantially more data to train effectively • More computationally demanding during both training and tuning • Prone to underfitting on small datasets due to model complexity
Contextual Limitation	• Underperformed when trained only on NIFTY50 or NIFTY200 datasets • Required full multi-stock data to achieve stable prediction patterns • Less effective compared to LSTM when only OHLCV inputs were available

5.3.4 Advantages and Disadvantages of Transformers

5.3.5 Role of Transformers in This Project

Transformers are the obvious next step to training with LSTM. Thus I wanted to check with this the effectiveness of transformers model in training for stock forecasting.

As significantly more data is required to train transformers model I only used all company data and NIFTY 200 data to check the effectiveness of Transformers in stock forecasting.

5.3.6 Trained Transformer Model Configurations

Transformer Model Without Indicators

The first model I trained with transformer only used the Raw features without any calculated indicators. It was trained with 60 epochs using batch size of 16.

Table 5.13: Model Architecture Summary

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 20, 5)	0
add (Add)	(None, 20, 5)	0
transformer_encoder (TransformerEncoder)	(None, 20, 5)	1,194
transformer_encoder_1 (TransformerEncoder)	(None, 20, 5)	1,194
global_average_pooling1d (GlobalAveragePooling1D)	(None, 5)	0
dropout_8 (Dropout)	(None, 5)	0
dense_6 (Dense)	(None, 64)	384
dropout_9 (Dropout)	(None, 64)	0
dense_7 (Dense)	(None, 1)	65

Transformer Model With Indicators

The overall model remains the same as without indicators but the number of features are increased, as the engineered features are also used with raw features. It was also trained for 60 epochs using a batch size of 16.

Table 5.14: Model Architecture Summary

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 20, 10)	0
add_1 (Add)	(None, 20, 10)	0
transformer_encoder_1 (TransformerEncoder)	(None, 20, 10)	3,124
transformer_encoder_2 (TransformerEncoder)	(None, 20, 10)	3,124
global_average_pooling1d (GlobalAveragePooling1D)	(None, 10)	0
dropout_13 (Dropout)	(None, 10)	0
dense_12 (Dense)	(None, 64)	704
dropout_14 (Dropout)	(None, 64)	0
dense_13 (Dense)	(None, 1)	65

5.3.7 Performance Summary

- Larger data sets are necessary for well generalized transformer model.
- Inclusion of Engineered Features, technical indicators, improve the stability and overall efficiency of the model.
- The model performs close to LSTM with indicators as the dataset used is still not as large that Transformers could create the margin, but also not so large that LSTM would overfit.

Table 5.15: Comparison of Recurrent Neural Network (RNN) Architectures

Model	Strength	Weakness	When to Use
GRU	Light & fast	May miss long-term patterns	Quick prototypes
LSTM	Strong memory (handles vanishing gradient)	Slightly slower, more complex architecture	Stable trends
Transformer	Handles long-range relations, scalable, parallelizable	Heavier, needs more data	Advanced / larger datasets

5.4 Conv1D + LSTM Hybrid Model

5.4.1 Introduction to the Hybrid Architecture

I wanted to combine the benefits of LSTM with the strengths of Convolutional neural networks (CNN) in time series forecasting.

The 1D CNN layer acts as a localized feature extractor similar to how humans try to find patterns in the candle stick data, such as hammer patterns, Doji, Inverted Hammer, Shootin Star etc. The CNN output can then be passed to LSTM that would convert them into temporal patterns.

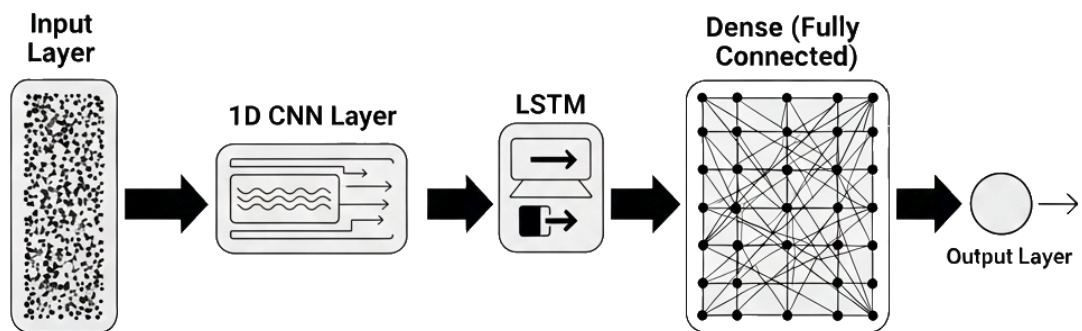


Figure 5.4: Architecture of the CNN + LSTM block.

5.4.2 Motivation for Using a Hybrid Model in Time-Series Forecasting

This hybrid design helps address:

- Capture local stock pattern with CNN
- Rapid fluctuation addressed
- Local trend dampened
- Smooth out features before they can be sent to LSTM

Table 5.16: Analysis of a Hybrid Deep Learning Model (Likely CNN-LSTM)

Category	Observations
Advantages	• Captures both short-range (CNN) and long-range (LSTM) dependencies • More resilient to noise than LSTM-only models • Can extract meaningful patterns even with moderate feature engineering • Performs well when trained on large multi-stock datasets
Disadvantages	• Requires more data than simple recurrent models • Higher computational cost and slower convergence • Hyperparameter tuning for convolution and LSTM layers increases complexity
Contextual Limitations	• Underperformed when trained on only NIFTY50/NIFTY200 datasets due to limited sequence diversity • Performed best when trained using the full multi-company dataset

Figure 5.5: Conceptual Diagram of a CNN-LSTM Model Architecture



5.4.3 Advantages and Disadvantages of the Hybrid Model

5.4.4 Trained Configurations of the Hybrid Model

Hybrid Model Without Indicators

The configuration of Raw features remain the same as the other models.

Hybrid Model With Indicators

The same indicators are included similar to LSTM.

Performance Summary

CNN LSTM models performed similarly to LSTM models in the most cases that I tested.

But when larger dataset were considered they generalized better than LSTM and

Table 5.17: Model Architecture Summary

Layer (type)	Output Shape	Param #
input_layer_5 (InputLayer)	(None, 15, 5)	0
conv1d_2 (Conv1D)	(None, 15, 64)	1,024
conv1d_3 (Conv1D)	(None, 15, 64)	12,352
lstm_3 (LSTM)	(None, 64)	33,024
dense_10 (Dense)	(None, 32)	2,080
dropout_11 (Dropout)	(None, 32)	0
dense_11 (Dense)	(None, 1)	33

Table 5.18: Model Architecture Summary (Likely CNN-LSTM)

Layer (type)	Output Shape	Param #
input_layer_7 (InputLayer)	(None, 15, 10)	0
conv1d (Conv1D)	(None, 15, 64)	1,984
conv1d_1 (Conv1D)	(None, 15, 64)	12,352
lstm_2 (LSTM)	(None, 64)	33,024
dense_14 (Dense)	(None, 32)	2,080
dropout_15 (Dropout)	(None, 32)	0
dense_15 (Dense)	(None, 1)	33

even transformers as they were able to capture short term patters and include them with the long term temporal patters as we combine CNN with LSTM.

Chapter 6

Results and Analysis

6.1 Overview

In this chapter I would like to present all the result that I have collected from all the models that were trained for this project. This includes all the chartes and observations that were collected and any interesting observations that I found for any particular models.

I have evaluated multiple machine learning architectures for forecasting next day closing prices of Indian stock market equities trading. The below mentioned models were assessed on both Raw Features as well as on combination of raw and engineered features. Evaluation of these models were done with error metrices and there are a lot of visual graphs that I have included for comparisons.

6.2 Training and Validation Behavior

- All models show loss curve decreasing during training.
- Confirming the ability to lean patterns in a meaningful way.
- How fast they converge depends on the dataset and features included.
- Using even basic features like only close data for stocks shows great impact on prediction showing the learning and pattern recognition ability of these models.

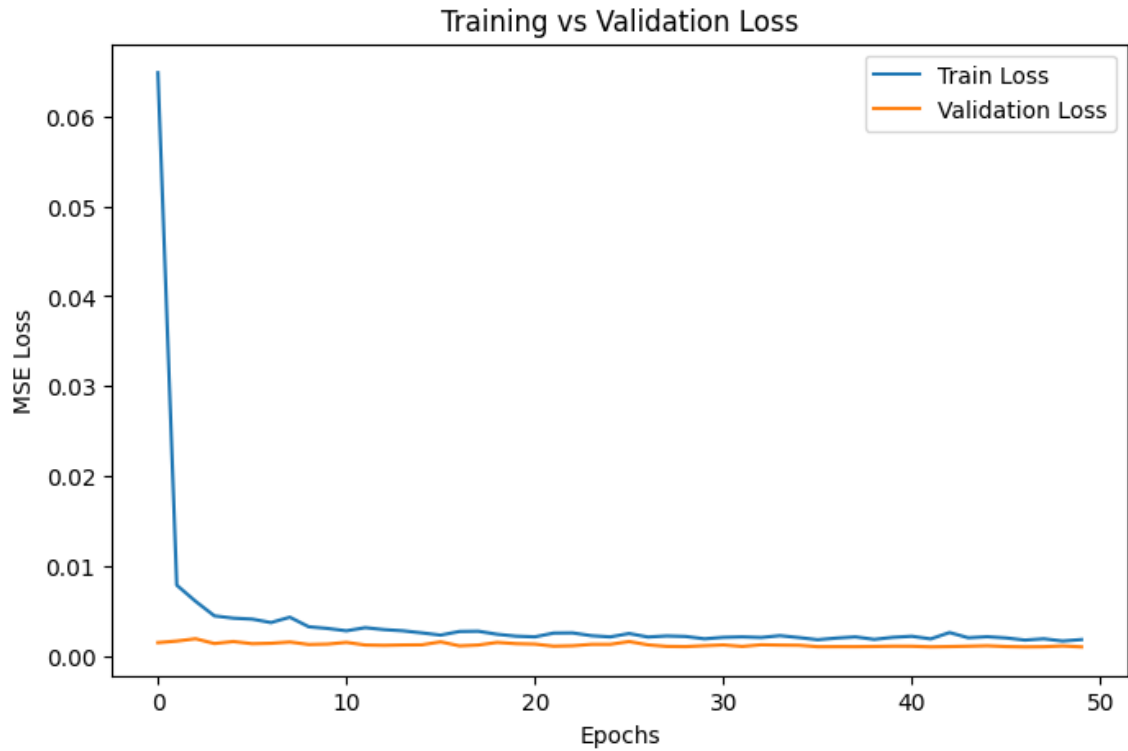


Figure 6.1: GRU without Indicators

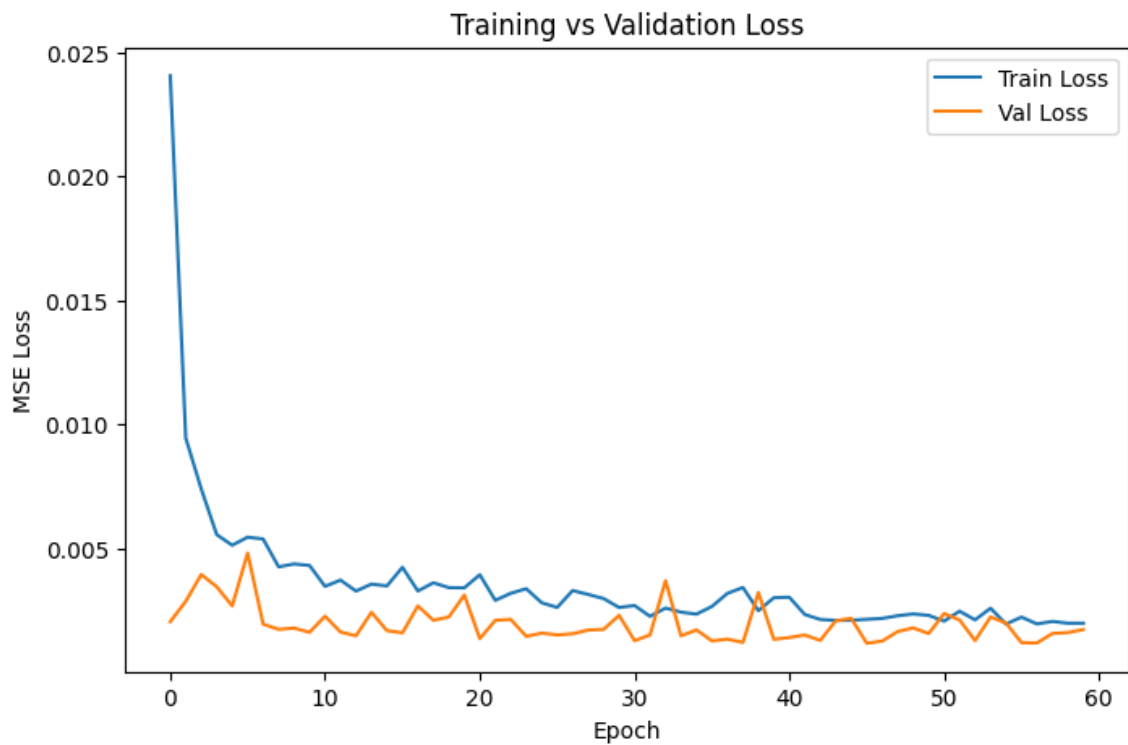


Figure 6.2: GRU with Indicators Validation Behaviour

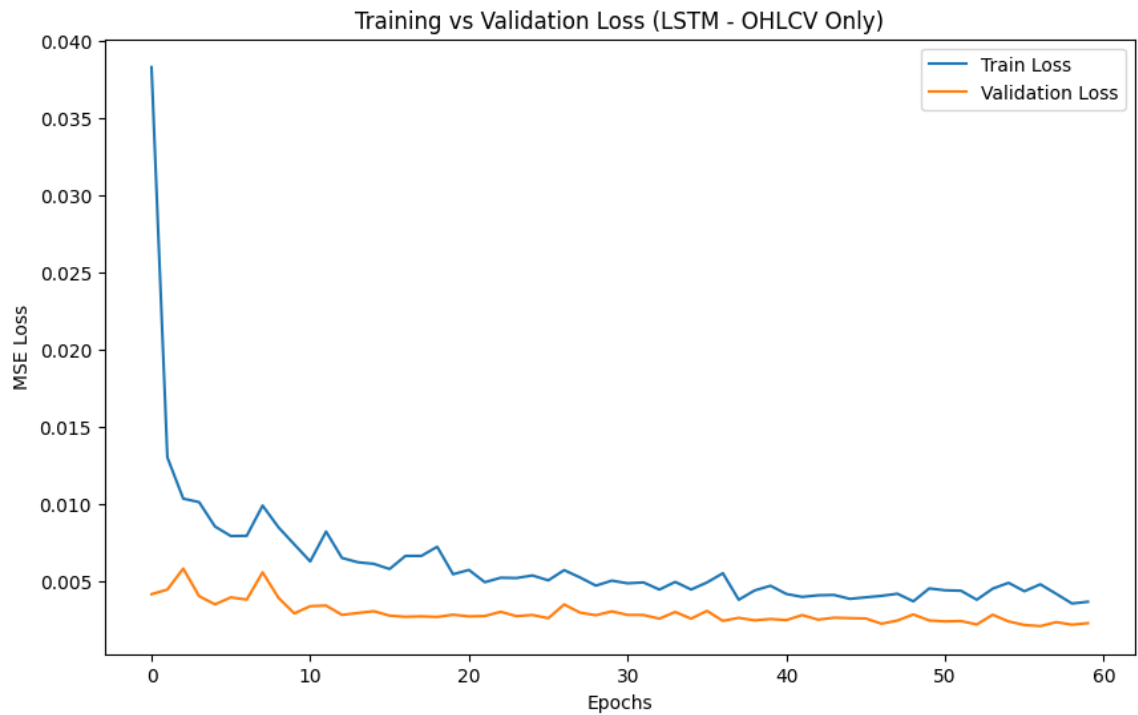


Figure 6.3: LSTM without Indicators Validation Behaviour

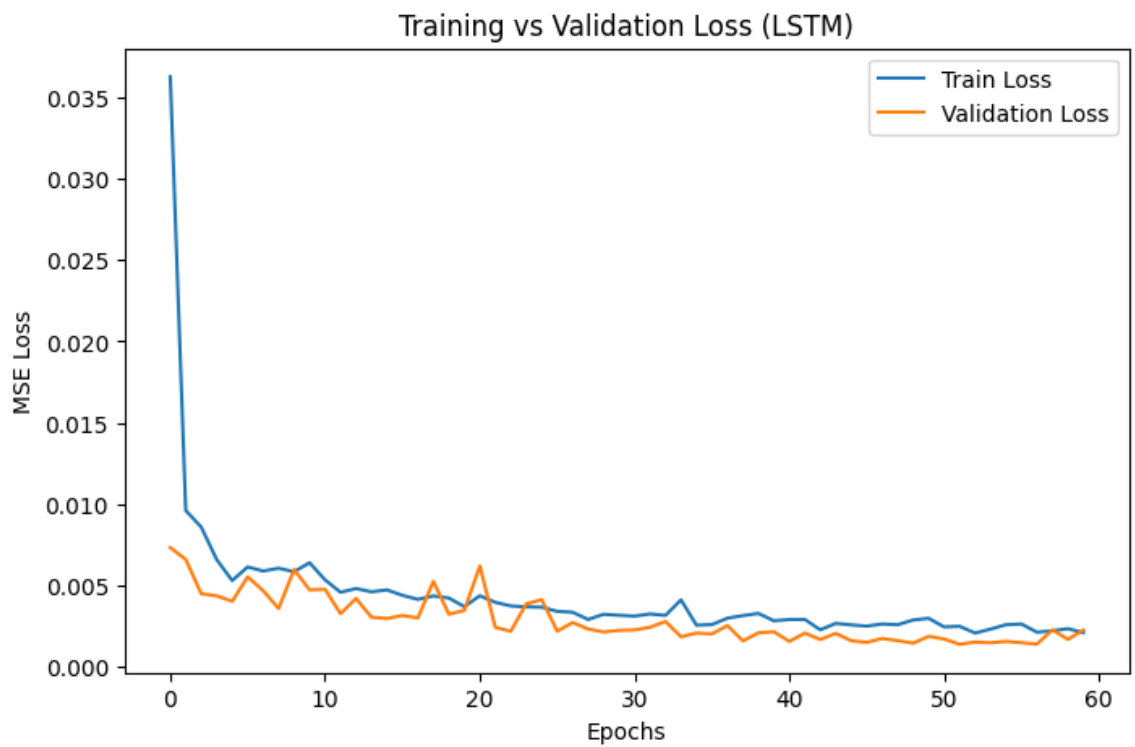


Figure 6.4: LSTM with Indicators Validation Behaviour

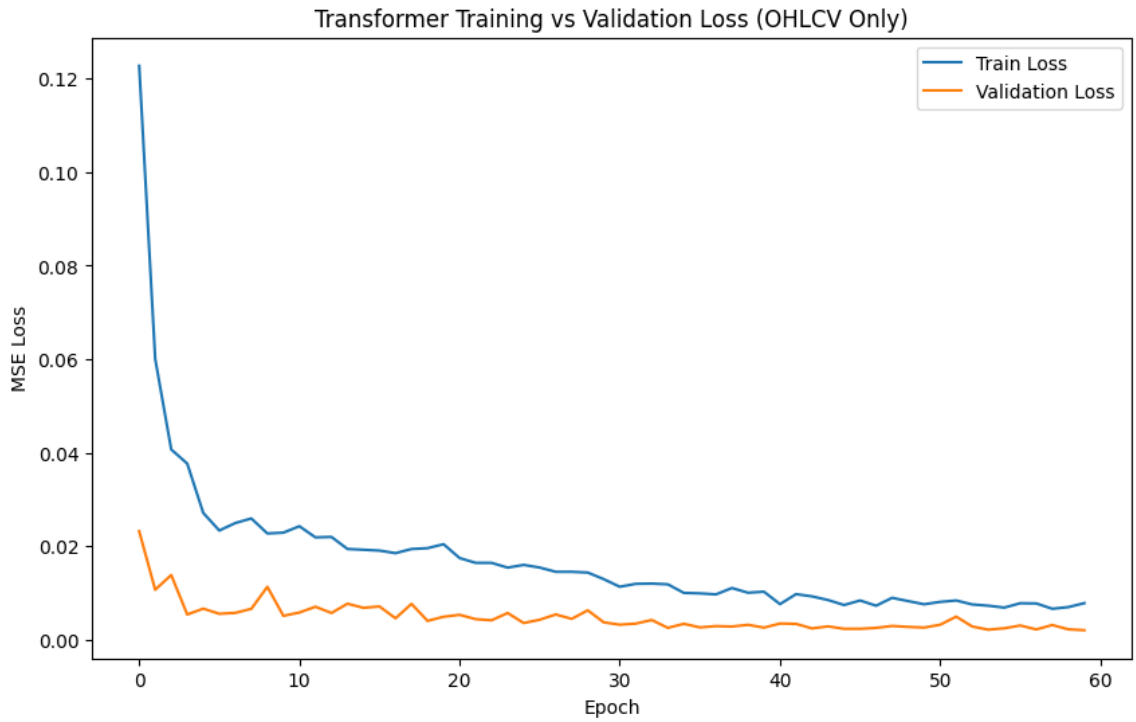


Figure 6.5: Transformers without Indicators Validation Behaviour

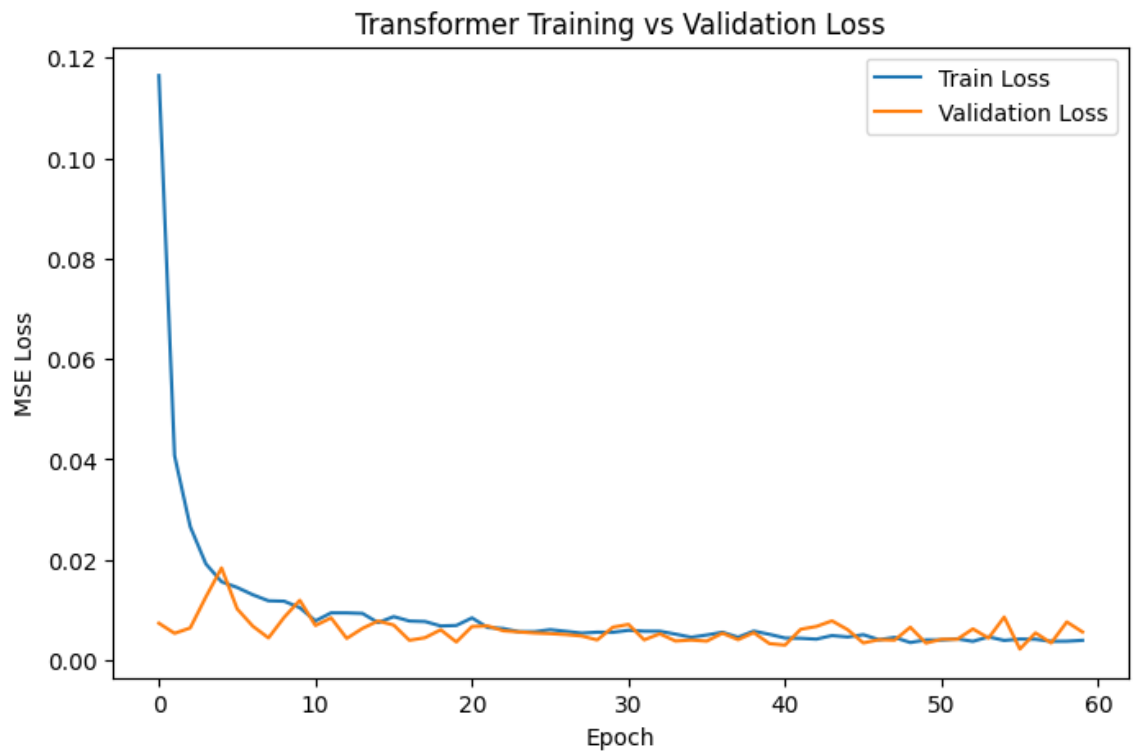


Figure 6.6: Transformers with Indicators Validation Behaviour

6.3 Single Company Data Used

Following are the observation that I got from training different models on single company data. These include both with and without indicator features used.

- For single-company datasets, **GRU and LSTM models** converged fastest and most consistently, likely due to their lower parameter count relative to Transformers and CNN-LSTM networks.
- When only the **closing price** was used as the input feature, models fit extremely well and often achieved very low training loss, indicating possible **overfitting**.
- Larger architectures such as **Transformer and CNN-LSTM** demonstrated slower convergence and required significantly more data to avoid underfitting.
- Models like Transformers and CNN-LSTM when used for single company data showed significantly worse performance than simpler algorithm like GRU, LSTM and even classic algorithms like linear regression.
- This shows a significant disadvantage in using such algorithms there, when the data-set is smaller.

Following figures demonstrate the above findings

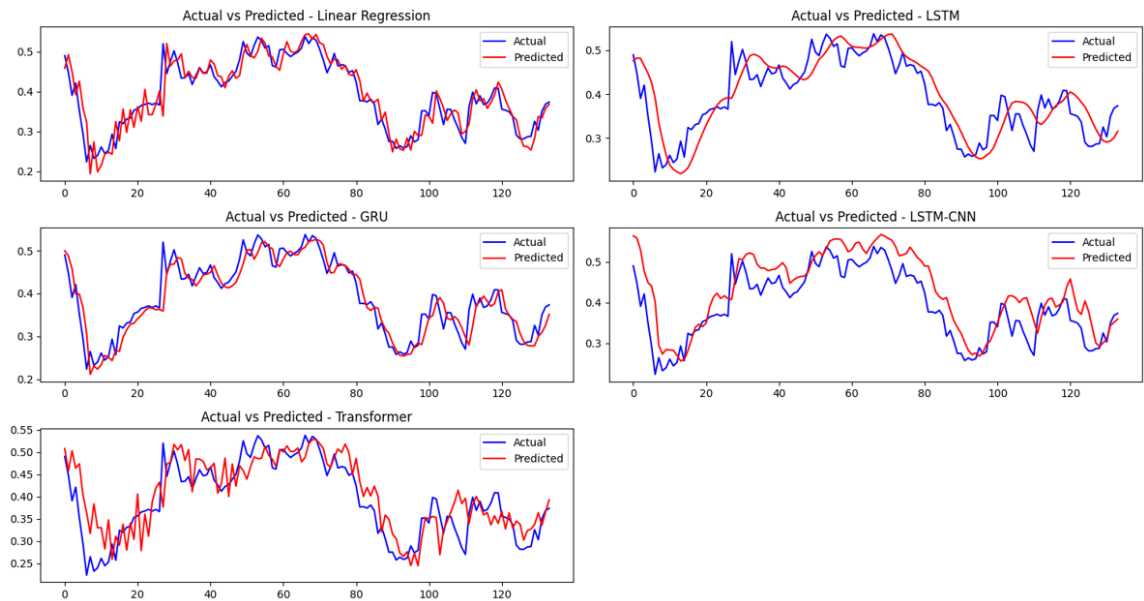


Figure 6.7: Regression Models on Single Company Data

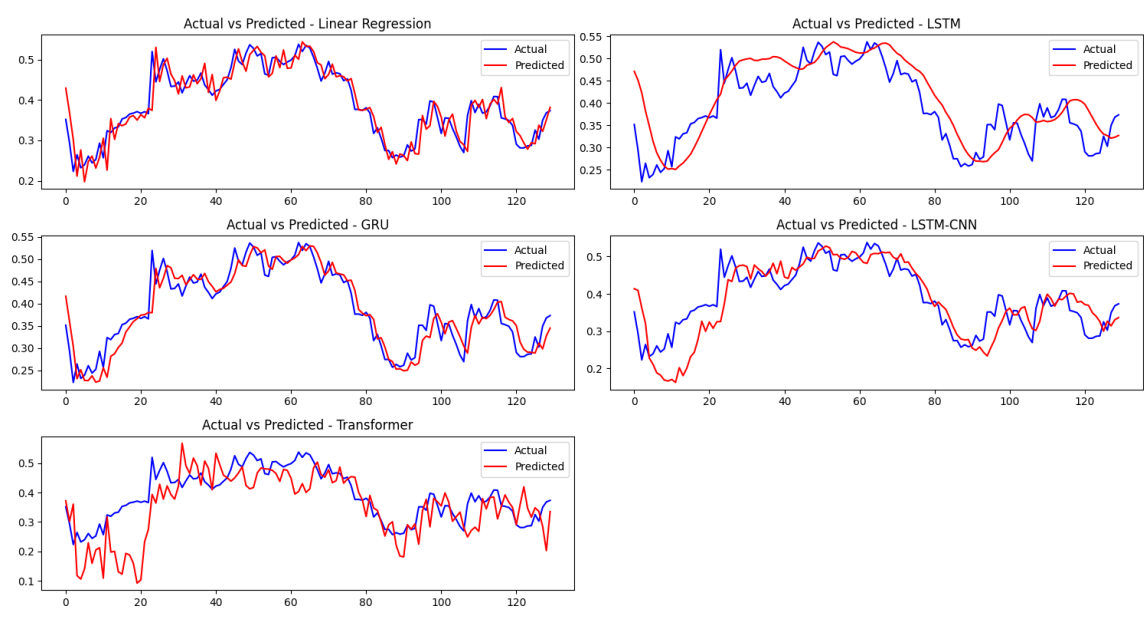


Figure 6.8: Regression Models on Single Company Data With Indicators



Figure 6.9: Regression Models Compared Without Indicators Included



Figure 6.10: Regression Models Compared with Indicators Included

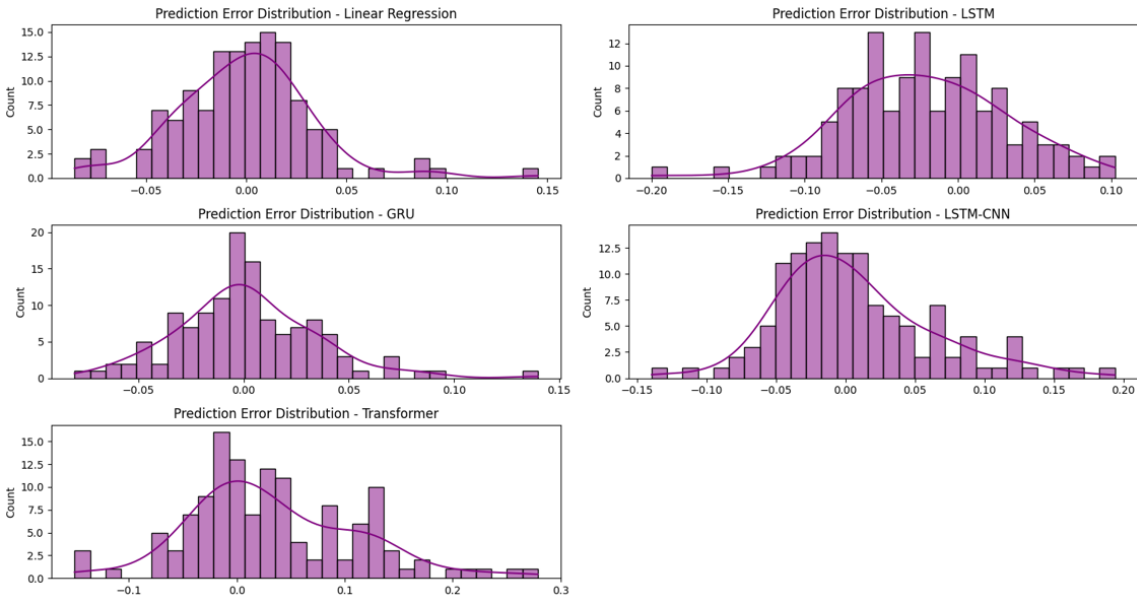


Figure 6.11: Prediction Errors for Regression Models on Single Company Data

6.3.1 Quantitative Evaluation Results

Table 6.1: Comparative Model Performance Across Datasets

Model Type	Single-Company Data	NIFTY50/200 Dataset	Full NSE Dataset
GRU	Best (fastest + accurate)	Poor generalization	Moderate
LSTM	Very good	Best generalization	Very good
CNN-LSTM	Underfits (small dataset)	Underfits (still limited)	Performs well, close to LSTM
Transformer	Underfits heavily	Weak performance	Performs well but not better than LSTM

The results clearly indicate that model complexity must match dataset size. Simpler recurrent models outperform complex architectures when data availability is limited.

6.4 Effect of Feature Engineering vs. Raw OHLCV Inputs

An important discovery in these experiments is that feature engineering did not always improve model performance.

Key conclusions:

- For single-company models, using only Close, Open, High, Low, Volume produced better results than using technical indicators, likely due to overfitting and noise sensitivity introduced by engineered features.
- For broader training datasets (NIFTY50, NIFTY200, or all NSE companies), adding indicators improved generalization and reduced error variance.

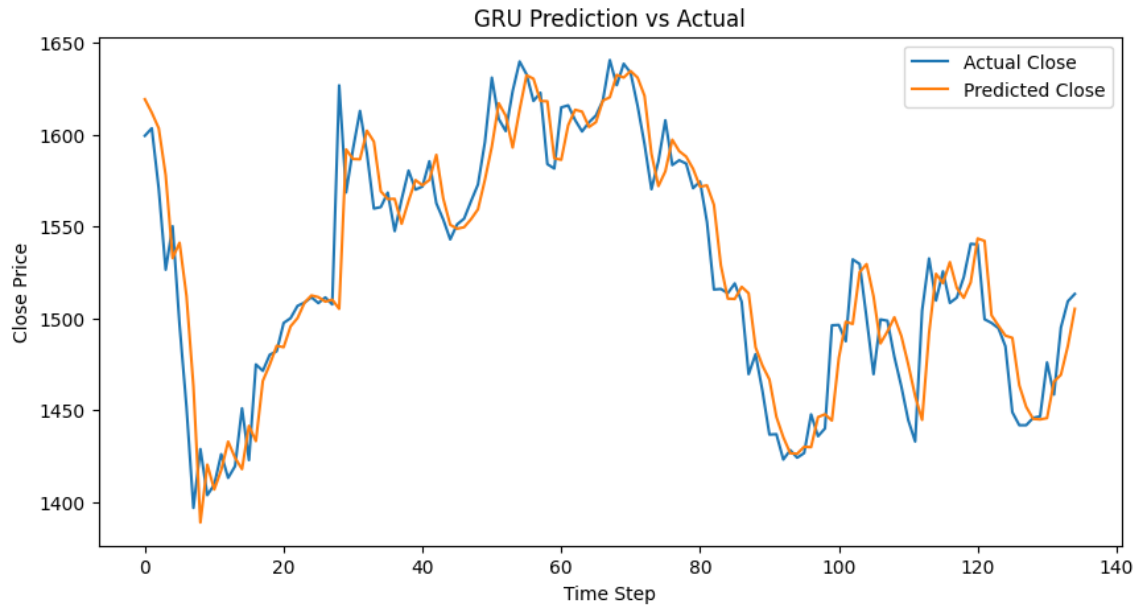


Figure 6.12: GRU with stock data

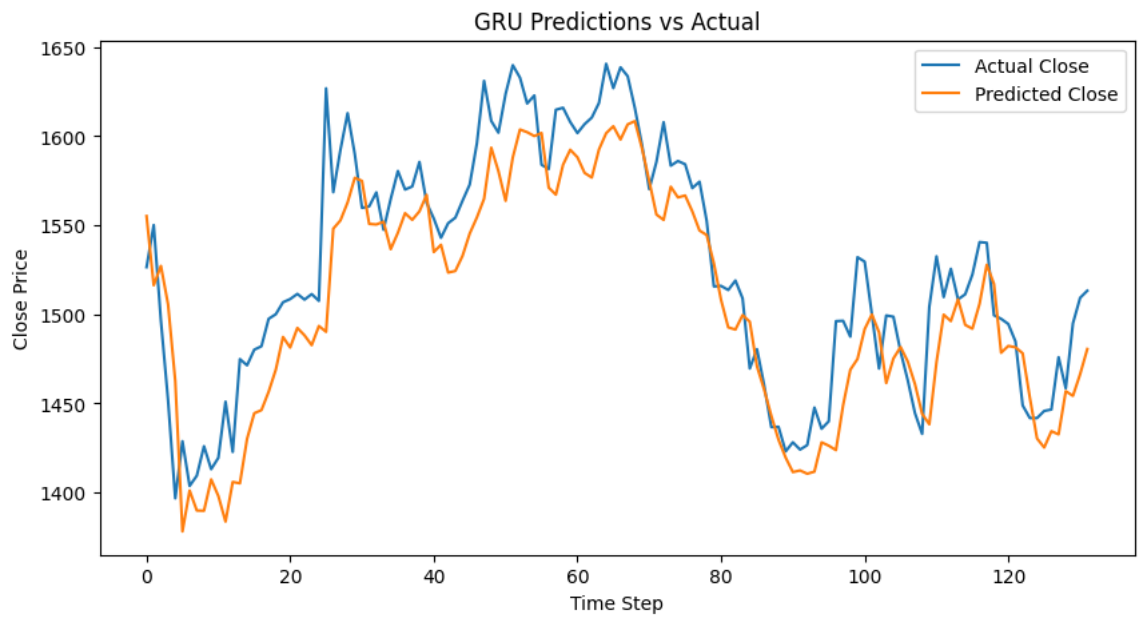


Figure 6.13: GRU with Indicators included (EMA (12, 26), RSI (14), ATR (14), OBV)

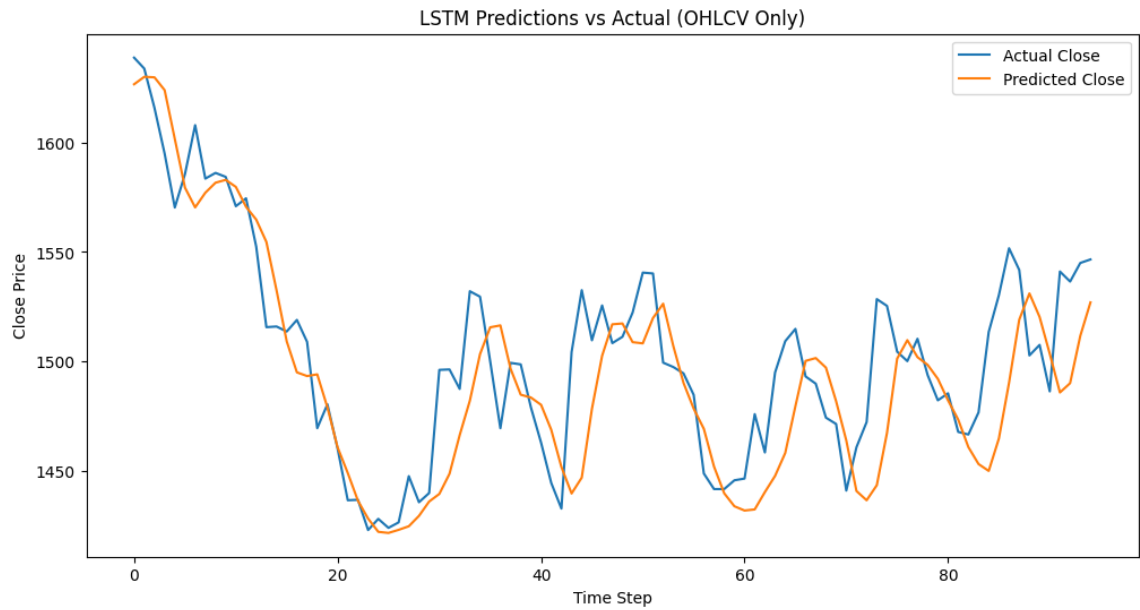


Figure 6.14: LSTM with stock data

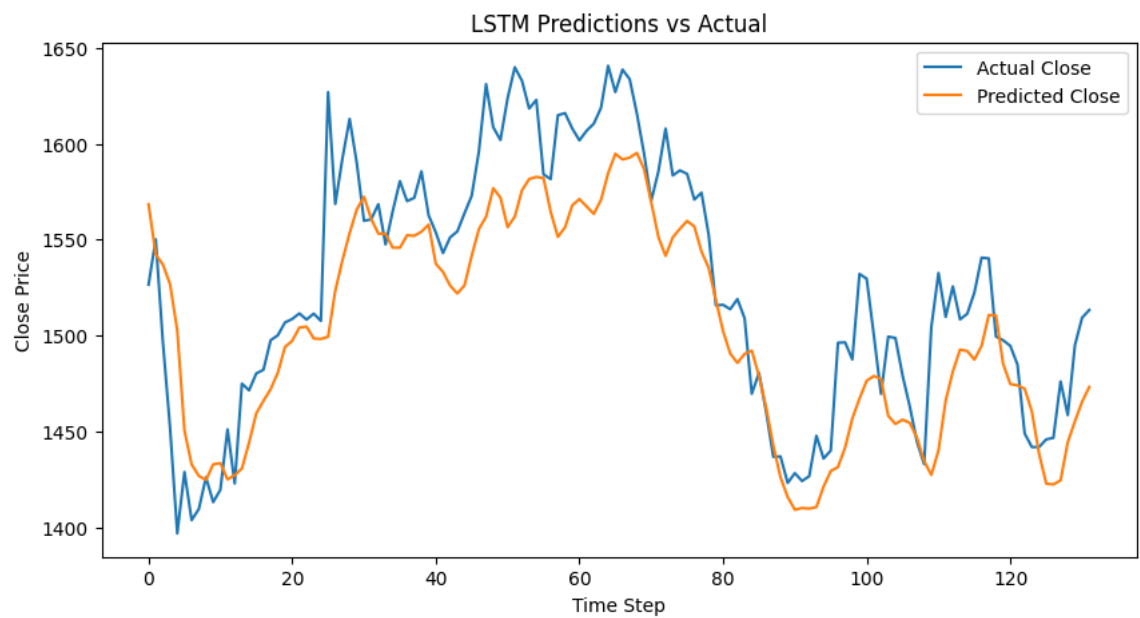


Figure 6.15: LSTM with Indicators (EMA (12, 26), RSI (14), ATR (14), OBV)

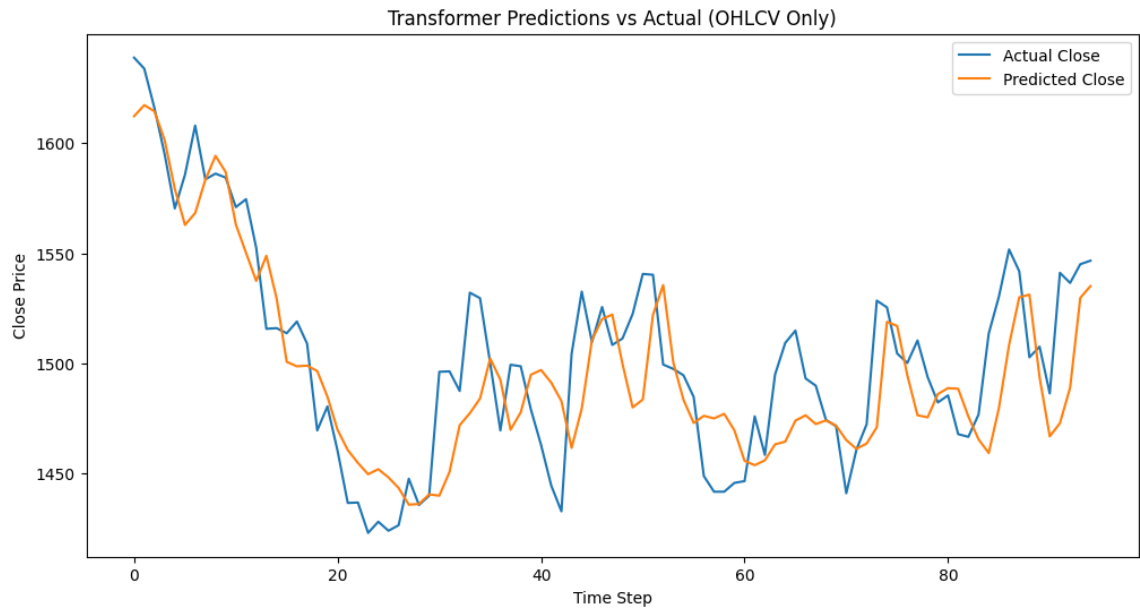


Figure 6.16: Transformers with stock data

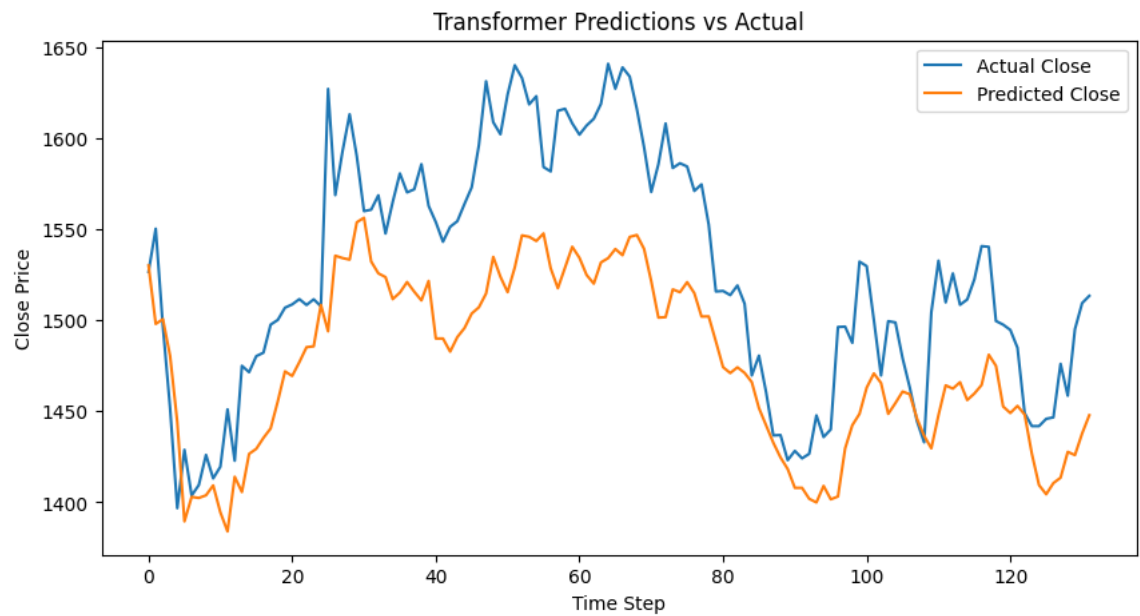


Figure 6.17: Transformers with Indicators (EMA (12, 26), RSI (14), ATR (14), OBV)

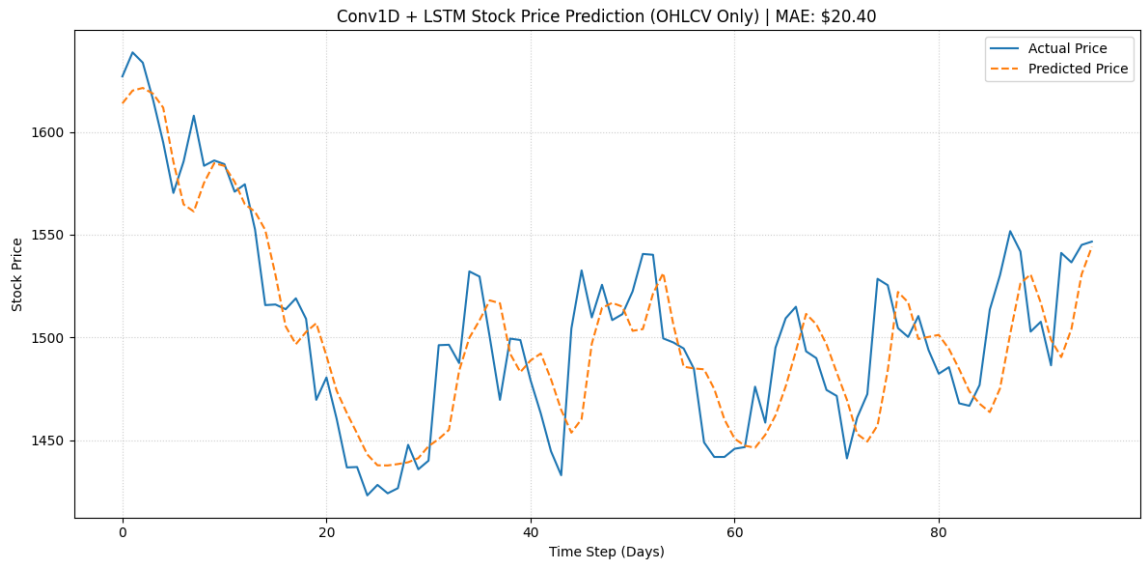


Figure 6.18: CNN-LSTM with stock data

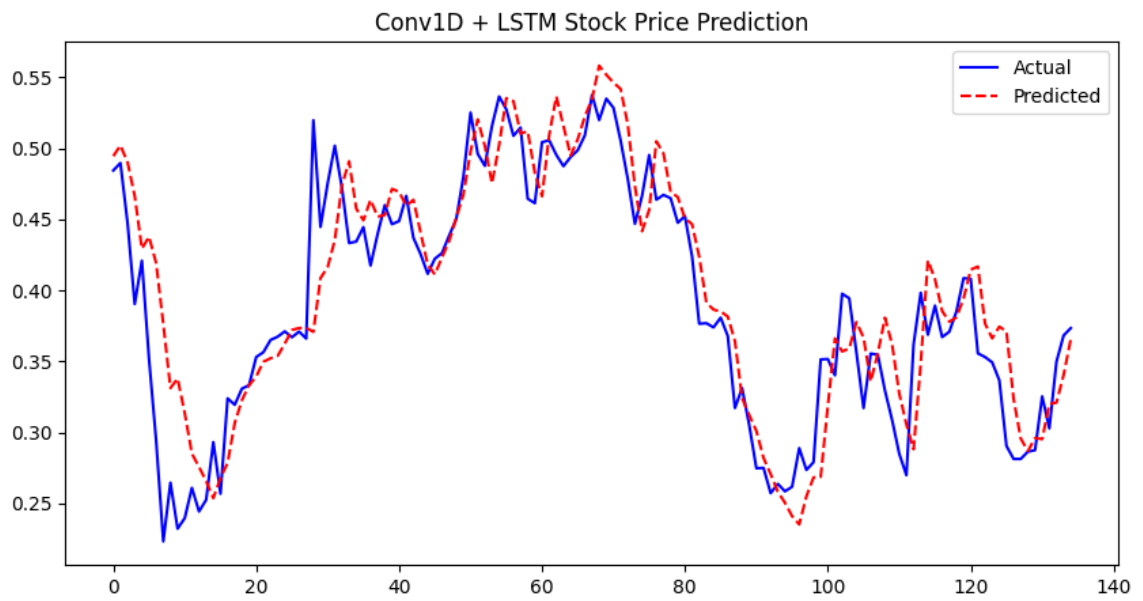


Figure 6.19: CNN-LSTM with Indicators (EMA (12, 26), RSI (14), ATR (14), OBV)

6.5 CNN-LSTM Feature Set Performance

For the CNN-LSTM architecture, three engineered feature combinations were tested:

Table 6.2: Feature Set Performance Comparison

Feature Set	Result
Close, MA5, MA20	Best performance
Close, EMA10, Standard Deviation	Moderate
Close return + STD20	Least consistent

This indicates that simple, trend-focused moving averages outperform more volatile or mathematically dense indicators in deep hybrid models.

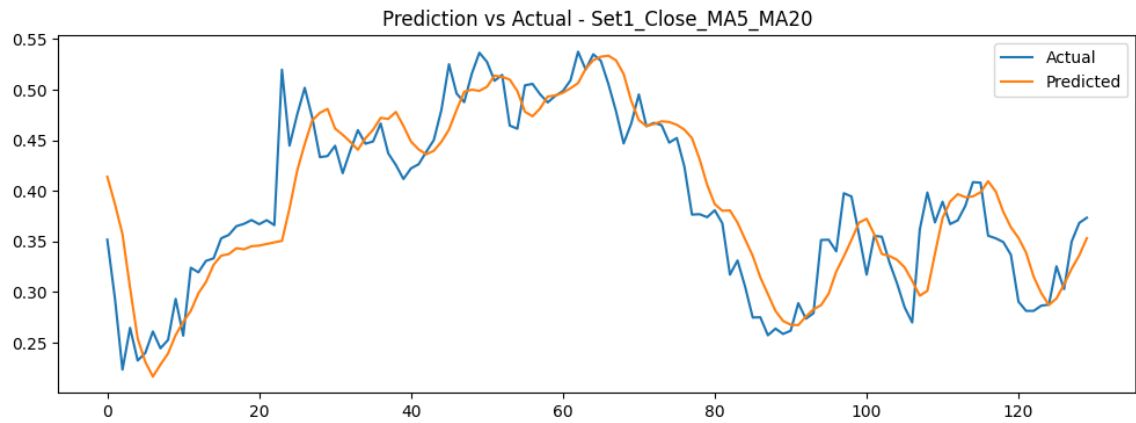


Figure 6.20: CNN-LSTM with Indicators (Close, MA5, MA20)

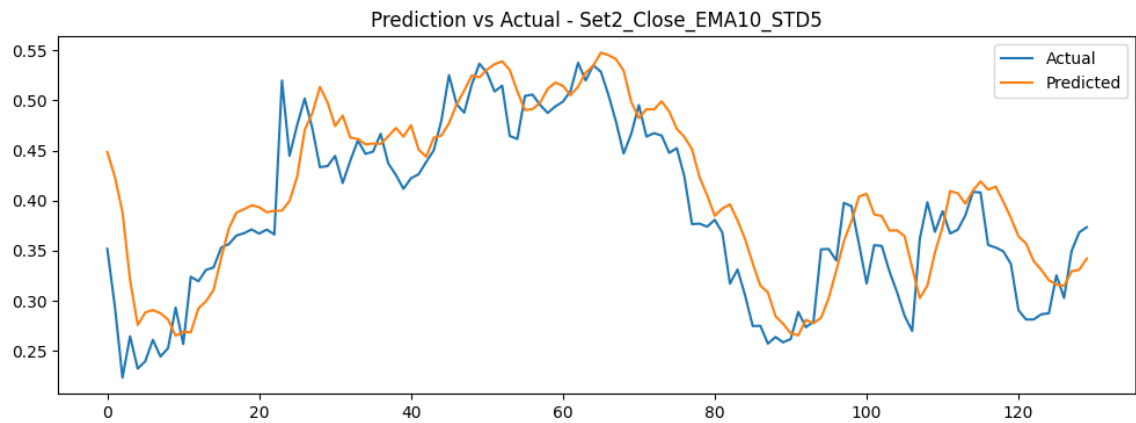


Figure 6.21: CNN-LSTM with Indicators (Close, EMA10, STD5)

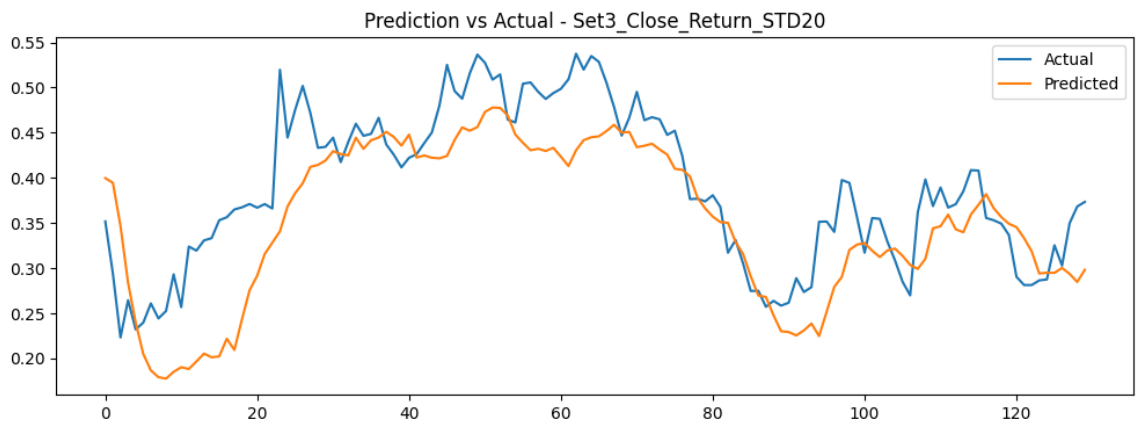


Figure 6.22: CNN-LSTM with Indicators (Close, Return, STD20)

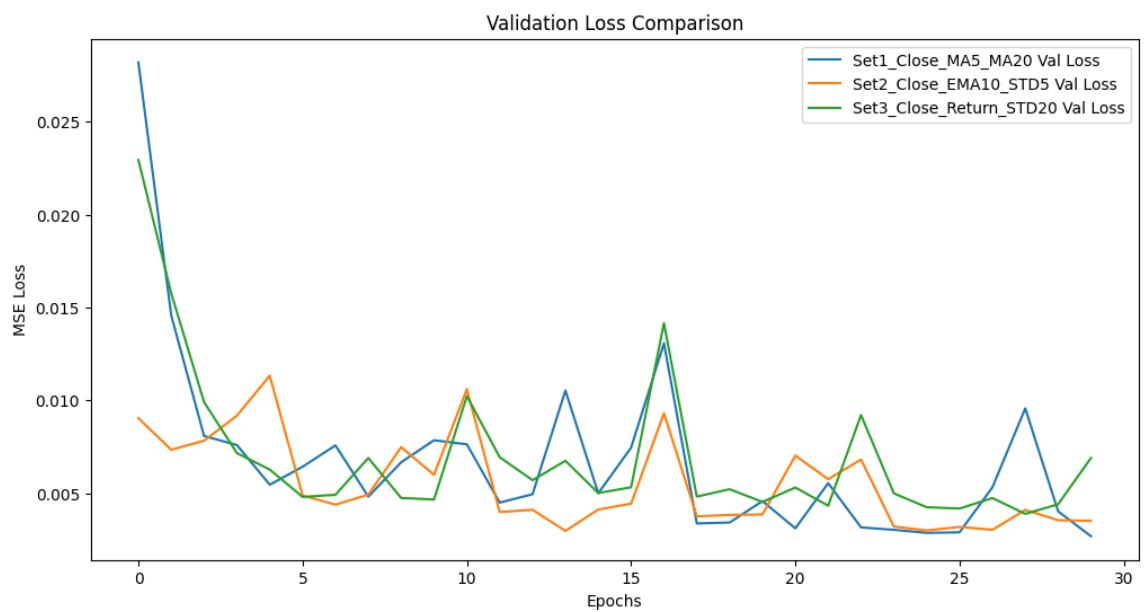


Figure 6.23: CNN-LSTM with Indicators Validation Loss Comparisons

6.5.1 Indicators tested and their purpose

Table 6.3: Strengths and Measures of Technical Indicator Sets

Indicator Set	What it Measures	Strengths
1. Close, MA5, MA20	Price trend over short (5-day) and medium (20-day) periods	Simple, easy to interpret; captures short- and medium-term momentum; identifies trend changes (crossovers)
2. Close, EMA10, STD	EMA10: weighted short-term trend; STD: price volatility	EMA reacts faster than simple MA; STD captures volatility (risk)
3. Close Return, STD20	Return captures percentage change , STD20 measures volatility of returns	Focuses on percentage movement rather than absolute price; STD20 captures medium-term volatility trends

Table 6.4: Summary of Technical Indicator Sets

Indicator Set	What it Measures	Usefulness in Stock Modeling
1. Close, MA5, MA20	Price trend over short (5-day) and medium (20-day) periods	Helps capture trend direction and short-term momentum , useful for regression or classification of up/down movement.
2. Close, EMA10, STD	EMA10: weighted short-term trend; STD: price volatility .	Useful to detect price momentum + risk ; volatility helps the model understand uncertainty and potential extreme moves.
3. Close Return, STD20	Return captures percentage change ; STD20 measures volatility of returns .	Good for risk-sensitive models and detecting high-volatility periods , can improve direction prediction.

6.6 Generalization vs. Specialization Behavior

- GRU and LSTM models that were trained on single company data, specialized very well to that companies prediction, but did lack in generalizing as other companies patterns could not be learnt.
- Models trained on larger data set containing many companies generalized extremely well but lacked in capturing the properties and patterns that any particular company's stock might exhibit.
- Models trained with NIFTY50 and NIFTY200 generalized well enought to the patterns of the bigger and more stable companies, but lacked when it came to pattern exhibited by smaller companies that could be more volatile.
- So a tradeoff it observed where smaller company specific models can perform well for that company and largers can capture the market patter overall.
- Thus I observed that having models according to the companies segment gives best result for that segment and cannot always be compared to other segments.
- This is also try for different industires like Banking, IT, manudacturing and others.
- It also means that each country's companies also behave in a different way to global events and how resistant thier market is to external factors.
- Thus country specific research is necessary when it comes to training models for that particular country.

Chapter 7

Summary and Conclusions

7.1 Summary of Work

In this project I have tried to research the Indian stock market behavior and tried to build a stock analysis system that would work for in accordance with the Indian stocks. This project focuses on building forecasting models that are specifically designed to understand and learn patterns from the Indian stock market.

For this I have tried to retrieve the most recent data for the Indian companies. To limit the scope of the project and make it comparable I chose NSE as the exchange and equity as the point of focus.

I used Kite connect API for its affordable price and generous limits on usage to get the most recent 2 years of data for all equities listed on NSE. This data was then used to train models. It is also used to get day to day data so that it could be added to the existing data of any stock in consideration and then train stock specific models on them.

And Finally a working demonstration of the complete project is created in python and the dashboard for visualization of the results is created in HTML and JS without using any framework.

7.2 Problem Statement and Solution

Problem Statement

- Create a stock market forecasting application, by training models on data available from Indian companies and compare the results with traditional statistical and manual technical analysis tools.

- Then validate these models on their accuracy, generalizable, and scaling ability, to find the best models possible for each usecase.

Gaps addressed by the Project

- Study and compare models on India specific dataset.
- Compare multiple different types of time series and hybrid models.
- Evaluate models on different size of data sets.
- Consider the impact of Indicators on the overall models performance
- Implement a demo to showcase the use of these models in trading.

7.3 Findings and thier Significance

- GRU models worked best when they were trained on limited data, mostly considering single company's data without using any indicators.
- This established the ability of GRU models to specialize well but they lack when it comes to generalizing well to large datasets.
- LSTM models showed the ability to train well on larger and smaller datasets, making them extremely versatile. They out performed most GRU, transformers and CNN LSTM in most cases.
- Transformer models even though gave good results when were trained on engineered features and large datasets, still did not give significantly high improvement over LSTM model trained on the same data.
- CNN-LSTM models out performed the LSTM models in very large data sets and when used with indicators but lacked with smaller datasets.
- CNN-LSTM models were exceptional in understanding local patterns as well as remembering the long term temporal patterns.
- Feature Engineering with indicators proved useful in enhancing the patterns when it was used with larger datasets.
- Indicators did not make a large difference when were considered with Raw features to train smaller models with GRU and LSTM especailly for single company data.

- NIFTY50 made of larger compnaies that are mostly stable and do not have extreme volatility and thus models trained with them did not generalize as well to the overall market patterns.
- NIFTY200 is composed of a diverse set of industries and companies and hence models trained with them got a large enough dataset that captured the overall market patterns much better.
- Models trained with all companies data performed well and generalized to the overall market behaviour but lacked as compared to specialized models trained on that companies data.

7.4 Limitations of the Project

- To limit the scope of the project limited data set was considered.
- Only daily data for past two years was considered.
- This project does not go into training for specific industries.
- Technincal indicators were kept constant and were not selected based on models or industries for optimal training.
- The models were only trained to predict next days closing and not trends.
- The models do not take into consideration any fundamental data or sentimental data.
- The project only focused on showing the prediction, but did not connect it to any automation system for re-enforcement learning

7.5 Future Work

Several opportunities exist to extend this research:

- Create models that are industry specific.
- Train models based on the company size, clubbing companies together based on their size.
- Increase the number of engineered features and indicators.

- Include models that are trained on and can predict values on Intraday trading.
- Refine hybrid models and find the best combinations and usecases for them.
- include Fundamentals of the company while training compnay specific models.
- Train models to understand fundamentals of the companies and use that as an input for stock prediction time series model.

7.6 Conclusion

To conclude this project and research undertook and was successful in determining the best time series models that could be trained on stock data for forecasting next days closing price. It also successfully demonstrates the use of and limitations of technical indicators that could be used as features for training models. As data from Indian companies was used, it also demonstrates the effectiveness of the models and indicators when it applies to the Indian stock market. I was also able to create a dash board that is useful as a tool to be used while trading, and also created a starting point for future improvement and feature additions.

Annexure

This annex provides supplementary material related to the datasets, model configurations, implementation details, and additional observations that support the work presented in this thesis on hybrid AI model driven technical analysis for stock forecasting.

A. Dataset and Preprocessing Summary

A.1 Datasets Used

- **Single-Company Dataset**

Used for detailed analysis of GRU, LSTM, Transformer, and CNN-LSTM performance on an individual stock. Data consists of daily OHLCV values and derived indicators over the most recent two-year window.

- **Index-Based Datasets (NIFTY50, NIFTY200)**

Used to train global models that learn cross-stock structure. Each constituent stock contributes a time series of daily OHLCV data with engineered technical indicators. These datasets are primarily used for LSTM, Transformer, and CNN-LSTM models trained on multi-stock data.

- **Broad NSE Universe Dataset**

Includes all available NSE-listed companies for which sufficient historical data is accessible. This dataset is mainly used for models that benefit from large-scale cross-sectional learning, such as Transformer and CNN-LSTM variants.

A.2 Preprocessing Pipeline (Summary)

The following steps summarize the data preprocessing pipeline implemented before training the models:

1. Data Cleaning and Ordering

- Parsing timestamps into a consistent datetime format.
- Sorting each stock's time series in chronological order.
- Skipping non-trading days (weekends and exchange holidays).

2. Missing Value Handling

- Forward filling for base price fields in rare missing cases where it does not distort temporal structure.
- Replacing missing volume values using a 7-day rolling average to better preserve liquidity patterns.
- Skipping stocks with extremely sparse or irregular trading histories.

3. Feature Engineering

- Calculation of Exponential Moving Averages (EMA), Relative Strength Index (RSI), Average True Range (ATR), and On-Balance Volume (OBV).
- Construction of lagged features and rolling volatility measures for capturing short-term memory and dispersion.

4. Scaling and Normalization

- Global Min–Max scaling applied per feature to enable multi-stock training.
- De-normalization applied at inference time using symbol-specific min–max values to recover prices in original scale.

5. Windowing Strategy

- Sliding window with a fixed lookback length (e.g., 20–30 days) to produce input sequences.
- The prediction target is the next-day closing price corresponding to each input window.

B. Model Configurations (Supplementary)

This section summarizes the key configurations of the deep learning models used in the project. The details complement the architecture descriptions presented in the main chapters.

B.1 Recurrent Models (GRU and LSTM)

- **Input Representation:** Sliding window of T days, each with OHLCV values and selected technical indicators.
- **GRU Model:**
 - One or more stacked GRU layers with hidden units in the range of 32–64.
 - Optional dropout between recurrent layers for regularization.
 - Fully connected output layer predicting the normalized next-day close.
- **LSTM Model:**
 - Stacked LSTM layers with comparable hidden sizes to GRU.
 - Emphasis on capturing longer temporal dependencies.
 - Dense regression head (one or more dense layers) mapping hidden representation to a scalar price forecast.
- **Optimization:** Adam optimizer with mean squared error (MSE) loss and early stopping based on validation loss.

B.2 Transformer-Based Model

- **Encoder-Style Architecture:**
 - Input sequences projected to a fixed embedding dimension.
 - Multiple self-attention encoder blocks, each comprising multi-head attention, feed-forward network, residual connections, and layer normalization.
- **Temporal Aggregation:**
 - Global pooling across time (e.g., average pooling) to obtain a fixed-length representation of the input window.
- **Prediction Head:**

- Dense layers with nonlinear activation.
- Final linear layer outputting the normalized closing price for the next day.

B.3 CNN–LSTM Hybrid Model

- **Convolutional Block:**

- One or more 1D convolutional layers to extract local patterns and short-term structures from the input sequence.
- Convolution filters slide over time, operating jointly on all feature channels.

- **Recurrent Block:**

- LSTM layers that process the convolved feature maps and integrate them over time.

- **Output Layer:**

- Dense layer mapping the LSTM output to a single normalized closing price prediction.

C. Implementation and API Summary

C.1 Software Stack and Environment

- **Backend:** Python for data acquisition, preprocessing, model training, and serving predictions.
- **Deep Learning Framework:** TensorFlow/Keras for implementing GRU, LSTM, Transformer, and CNN-LSTM architectures.
- **Frontend:** HTML and JavaScript used to visualize time-series charts, recent price tables, and model predictions via a lightweight web interface.
- **Data Source:** Historical OHLCV data retrieved from exchange APIs and stored as CSV files organized by symbol.

C.2 Prediction and Evaluation Workflow

1. Symbol Selection:

- User selects a stock (e.g., from NIFTY50 or NIFTY200) via the web interface.
- Backend loads the corresponding historical CSV file and applies the same preprocessing pipeline used during training.

2. Model Selection:

- The interface exposes a dropdown allowing selection among trained models (e.g., GRU, LSTM, Transformer, CNN-LSTM).
- The backend routes the preprocessed sequence through the chosen model.

3. Forecast Generation:

- Next-day closing price is predicted in normalized space.
- The prediction is then de-normalized back to the actual price scale for the chosen symbol.

4. Visualization and Error Analysis:

- The predicted and actual series for the recent evaluation window (e.g., last three months) are plotted together.
- Error metrics such as Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) are computed and displayed.

C.3 Practical Considerations

- **Training Stability:**

- Early stopping is used to prevent overfitting when validation loss stops improving.
- Learning rates and batch sizes are chosen empirically based on stability and convergence speed.

- **Generalization vs. Specialization:**

- Global models trained on multi-stock data (e.g., NIFTY200) capture common structural patterns across symbols.
- Per-symbol models trained on a single company often specialize better for that stock but may not generalize to others.

- **Limitations:**

- Performance can degrade during extreme market events or structural breaks not present in historical training data.
- Models are trained purely on technical inputs; they do not account for macroeconomic variables or textual news sentiment in the current implementation.

This annex is intended to serve as a concise technical reference for the data, models, and implementation details underlying the experimental results presented in the main body of the thesis.

References

- [1] F. Huang, S. Huang, and A. Huang, “Predictive modeling of the global stock markets using machine learning,” *Journal of Financial Risk Management*, 2022.
- [2] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [3] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, “Learning phrase representations using rnn encoder–decoder for statistical machine translation,” in *Proceedings of EMNLP*, 2014.
- [4] Y. Smarte and G. D. Ho, “Review of machine learning for stock prediction based on technical analysis,” *IEEE Transactions on ...*, 2021.
- [5] H. Zhu, *Stock prediction using artificial intelligence technology: A review*, ResearchGate, 2025.
- [6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, 1998.
- [7] F. Jin, “Cnn prediction of the stock market indicators using a hybrid deep learning system,” *IEEE Transactions on ...*, 2024.
- [8] A. Vaswani et al., “Attention is all you need,” in *Proceedings of NIPS*, 2017.
- [9] F. Khanpuri, S. Khanpuri, and A. Khanpuri, “Sentiment analysis on fundamental data to predict stock prices,” *EAI Endorsed Transactions on AI and Robotics*, 2024.
- [10] F. Huang, S. Huang, and A. Huang, “Deep learning to llms: A survey on quantitative investment,” *IEEE Transactions on ...*, 2025.
- [11] F. Cao, S. Cao, and A. Cao, “Applying llms for stock forecasting,” *Proceedings of ICAIF ’24*, 2024.
- [12] F. Cilingiroglu, *How sentiment is changing the stock price prediction market*, Forbes, 2025.

- [13] F. Balasubramanian, S. Balasubramanian, and A. Balasubramanian, “Stock market analysis using biometric indicators,” *SHS Web of Conferences*, 2024.
- [14] F. Mokhtari, S. Mokhtari, and A. Mokhtari, *The impact of artificial intelligence in enhancing predictive stock trading*, Munich Personal RePEc Archive, 2021.
- [15] A. Gupta, “The role of ai in the stock market to enhance predictability & strategy,” *IJEER*, 2024.